CyberSight DW: A Multi-Backend Data Warehouse for Cybersecurity Threat Intelligence

YASIN HESSNAWI

Vladimir Zadorozhny

University of Agder, 2026
Faculty of Engineering and Science
Department of Information and Communication Technology

Master

Mandatory Individual Declaration

The student is responsible for becoming familiar with what constitutes permitted aids, the guidelines for their use, and the rules concerning citation. This declaration is intended to make the student aware of that responsibility and of the consequences of cheating. Failure to submit the declaration does not absolve the student of these obligations.

1.	I hereby declare that this submission is my own work and that I have not used sources or received assistance other than those stated in the submission.	Yes
2.	**I further declare that this submission:** • Has not been used for another examination at another department, university, or college in Norway or abroad. • Does not refer to the work of others without such references being stated. • Does not refer to my own earlier work without such references being stated. • Lists all references in the bibliography. • Is not a copy, duplicate, or transcription of another's work or submission.	Yes
3.	I understand that a breach of the above is considered cheating and may result in the annulment of the examination and exclusion from universities and colleges in Norway, in accordance with the Act relating to Universities and University Colleges §§4-7 and 4-8 and the Regulations on Examinations §§ 31.	Yes
4.	I understand that all submitted work may be subject to plagiarism checks.	Yes
5.	I understand that the University of Agder will handle all suspected cases of cheating in accordance with the institution's regulations for such cases.	Yes
6.	I have familiarised myself with the rules and guidelines on the use of sources and references available on the library's web pages.	Yes
7.	This is a solo project; the group-effort clause concerning individual assessment does not apply.	N/A

Abstract

This report presents CyberSight DW, a multi-backend data warehouse for cybersecurity threat intelligence built on the CICIDS 2017 network intrusion dataset. A single cleaning pipeline ingests 2,099,971 cleaned flow records through a Kafka streaming layer and writes every event simultaneously to three database backends: a PostgreSQL star schema with pre-aggregated summary tables, a MongoDB document store with embedded sub-documents, and a Neo4j property graph in which aggregated edge properties carry pre-computed counts. A Streamlit dashboard reads from all three backends and exposes twelve decision-support queries alongside a side-by-side backend comparison, with a ksqlDB layer providing tumbling-window tables for live panels on the Overview page.

The three backends occupy different points on a pre-computation spectrum. PostgreSQL and MongoDB compute aggregates at query time over raw events, with PostgreSQL benefiting from normalisation and summary tables while MongoDB benefits from embedding and join-free single-document reads. Neo4j shifts aggregation to write time, storing only aggregated relationships, which makes graph-native queries (co-attacking IP pairs) cheap but prevents it from answering per-event questions such as average flow duration. The backend comparison therefore reads each model on its own terms rather than against a uniform benchmark, which would hide rather than reveal the architectural trade-offs.

The report covers the star schema design with full DDL, the six pre-aggregated summary tables with their batch-job SQL, the two NoSQL alternative implementations, the ten-service Docker topology, the streaming pipeline with at-least-once delivery and dead-letter queue handling, and the measured outcomes of a full end-to-end run. The closing comparison positions the three models as complementary rather than competing: PostgreSQL for multi-dimensional aggregation, MongoDB for event-centric flexibility, and Neo4j for relationship queries that a table cannot express cheaply.

Contents

Abstract **ii**

List of Figures **vii**

Acronyms and Terminology **ix**

List of Tables **xi**

1 Introduction **1**

1.1 Motivation and Problem Context 1

1.2 Scope and Objectives 1

1.3 System Overview 2

1.4 Users and Roles 3

1.5 Report Structure 4

2 Background and Assumptions **5**

2.1 Data Warehousing Fundamentals 5

2.2 Streaming and Alternative Models 5

2.3 Assumptions 6

3 Data and the ETL Pipeline **8**

3.1 Dataset Description 8

3.2 Cleaning Pipeline 9

3.3 Event Message Schema 10

3.4 ETL Flow and Loading 11

4 Relational Data Warehouse Design **13**

4.1 Schema Overview 13

4.2 Dimension Tables 14

4.3 Fact Table 16

4.4 Loading the Star Schema 16

4.5 Pre-Aggregated Summary Tables 17

4.6 Batch Job SQL 18

5 NoSQL Implementations **20**
5.1 Approach and Structure . . . 20
5.2 MongoDB Implementation . . . 20
5.3 Neo4j Implementation . . . 22
5.4 Comparing the Two NoSQL Shapes . . . 24

6 Infrastructure and Streaming **25**
6.1 Docker Topology . . . 25
6.2 Streaming Pipeline . . . 26
6.3 Delivery Guarantees and Failure Handling . . . 27

7 Queries, Frontend, and User Scenarios **29**
7.1 The Twelve Decision-Support Queries . . . 29
7.2 The Streamlit Dashboard . . . 30
7.3 User Scenarios . . . 31

8 Results and Verification **32**
8.1 End-to-End Run . . . 32
8.2 Data Volume and Integrity . . . 32
8.3 Warehouse Statistics . . . 33
8.4 Attack and Geographic Distribution . . . 33
8.5 Observed Query and Write Performance . . . 34

9 Relational vs NoSQL Comparison **36**
9.1 Framing the Comparison . . . 36
9.2 Dimension-by-Dimension Comparison . . . 36
9.3 Query Suitability . . . 37
9.4 Advantages and Disadvantages . . . 38
9.5 Synthesis . . . 39

10 Conclusion **40**
10.1 Summary of Findings . . . 40
10.2 Limitations . . . 40
10.3 Closing . . . 40

Bibliography **41**

A Full SQL Listings **42**
A.1 Additional Summary Table DDL . . . 42
A.2 Additional Batch Scripts . . . 43
A.3 Dimension Seed Data . . . 44

A.4 ksqlDB Initialisation 46
A.5 Decision-Support Query Reference SQL 47

B AI Declaration **51**

List of Figures

1.1 System architecture of CyberSight DW 3

3.1 ETL flow from CSV to loaded backends 12

4.1 Star schema of CyberSight DW 13

5.1 Neo4j graph model for CyberSight DW 22

6.1 Data flow across the Docker services 25

Acronyms and Terminology

The terms below appear most often in the report. The list is deliberately compact and excludes terms whose meaning is unambiguous in context. Where a term has both a precise technical meaning and a looser colloquial one, the definition given here is the one used in the project.

Table 1: Glossary of acronyms and technical terms used in this report.

Term	**Definition**
ACID	Atomicity, Consistency, Isolation, Durability. The set of transactional guarantees traditionally offered by relational databases.
BSON	Binary JSON. The binary-encoded document format MongoDB uses internally to store and transmit its records; adds type fidelity and compactness over plain JSON.
CICIDS 2017	Canadian Institute for Cybersecurity Intrusion Detection System 2017 dataset. A public dataset of labelled network flow records used as the input for this project.
CSV	Comma-separated values. The file format in which the source dataset is distributed.
Cypher	The query language used by Neo4j for pattern matching over property graphs.
DDL	Data Definition Language. The subset of SQL concerned with declaring schema (`CREATE`, `ALTER`, `DROP`).
Dimension table	A table that stores the contextual attributes of a fact, such as time, source, or attack type.
DLQ	Dead-letter queue. A Kafka topic to which the consumer redirects messages it cannot process, so that they can be inspected later rather than being lost or blocking the pipeline.
DW	Data warehouse. A repository optimised for analytical queries over consolidated historical data, as distinct from an operational database.
ETL	Extract, Transform, Load. The pipeline that moves data from source systems into the warehouse while cleaning and reshaping it.
Fact table	The central table of a star schema, one row per observed event, with foreign keys to the dimension tables and numeric measure columns.
INET	PostgreSQL data type for IPv4 and IPv6 addresses. Validates the value on insert and supports subnet-mask arithmetic, which makes it the natural type for a source- or destination-IP column.

Continued on next page

Table 1 continued from previous page

Term	Definition
JSON	JavaScript Object Notation. The text-based serialisation format used for Kafka messages between the producer and the consumer in this project.
Kafka	Apache Kafka, a distributed log-based message broker. Used here as the transport layer between the producer and the consumer.
KPI	Key Performance Indicator. A headline metric displayed prominently on the dashboard, for example total events, attack events, or attack rate.
ksqlDB	A streaming SQL engine on top of Kafka. Used here for tumbling-window views that the dashboard reads as real-time panels.
MERGE (Cypher)	A Neo4j clause that matches a pattern if it exists and creates it otherwise, similar in spirit to an SQL upsert.
MongoDB	A document-oriented database used as one of the two NoSQL backends in this project.
Neo4j	A property graph database used as the second NoSQL backend. Stores nodes, relationships, and properties on both.
NoSQL	A family of non-relational databases that relax or replace the tabular model, for example document stores, key-value stores, column-family stores, and graph databases.
OLAP	Online Analytical Processing. The query pattern a data warehouse is optimised for: aggregate, slice, drill down.
SOC	Security Operations Center. The team within an organisation that monitors network activity and responds to security events.
Star schema	A dimensional modelling pattern with one fact table in the centre and dimension tables radiating out, all joined directly to the fact.
Streamlit	A Python library for building data dashboards. Used as the front end of this project.
Surrogate key	A synthetic primary key, usually an integer, used instead of a natural business key to keep joins narrow and to insulate the schema from changes in source data.
Summary table	A pre-aggregated table derived from fact and dimension data, populated by a batch job and read by the dashboard to avoid scanning the full fact table on every request.

List of Tables

1 Glossary of acronyms and technical terms used in this report. ix

2.1 Explicit assumptions the project rests on. 6

3.1 CICIDS 2017 source files used in this project. 8

3.2 Cleaning steps applied by the producer before each chunk leaves for Kafka. . 10

4.1 Pre-aggregated summary tables, their grain, and the queries they back. . . . 17

5.1 MongoDB and Neo4j represent the same data through opposite philosophies. 24

7.1 Decision-support query catalogue. Each query is implemented across the backends that can answer it naturally. 29

8.1 Data quality metrics after a full end-to-end run. 32

8.2 PostgreSQL star schema and summary tables. 33

8.3 MongoDB collections. 33

8.4 Neo4j graph: nodes and relationships after `compute_co_attackers`. 33

8.5 Event distribution by attack category. Severity is the assigned rating from the cleaning pipeline's mapping module. 34

8.6 Observed write throughput per backend during streaming. 34

9.1 Backend comparison matrix. Qualitative labels reflect relative behaviour on this dataset and hardware profile. 37

9.2 Query suitability by backend. "Native" means the backend answers the question through its natural idiom; "Partial" means it answers but with fewer columns or an approximation; "–" means the query is not implemented for that backend. 38

Chapter 1

Introduction

1.1 Motivation and Problem Context

Cybersecurity teams work with streams of network traffic that are, in the raw, almost unreadable. A single mid-sized organisation generates millions of flow records per day, each carrying source and destination addresses, port numbers, packet counts, byte counts, protocol codes, and a handful of derived rate measures. Buried in that flood are the events that actually matter: brute-force attempts, denial-of-service spikes, lateral movement across internal hosts, and the quieter signals of infiltration campaigns. The gap between having the data and being able to act on it is a data warehousing problem, and it is a harder one than the textbook retail example because the interesting questions cross several axes at once. Analysts need to know what types of attacks are happening, who is behind them, and when they peak, often in the same breath.

The IKT553 project brief asks for a relational data warehouse with a star schema, two alternative implementations on NoSQL platforms, dockerised infrastructure, a Kafka streaming layer, and a comparison of the approaches. The learning objectives are concrete: practise multidimensional modelling on a non-trivial dataset, examine how relational warehousing concepts translate into document and graph models, and work through the operational plumbing that connects ingestion to analytics. Cybersecurity threat intelligence turned out to be a good fit for those objectives. The data is structured enough for a star schema to make sense, rich enough for document embedding to pay off, and relational enough between attacker and target entities for a graph model to do work that a table cannot.

1.2 Scope and Objectives

The project takes a public network intrusion dataset, CICIDS 2017,[1] and pushes it through a single ingestion pipeline that populates three different database backends in parallel. PostgreSQL holds the canonical star schema along with pre-aggregated summary tables. MongoDB holds the same events as embedded documents that fold the star-schema context into each row. Neo4j holds an aggregated property graph whose edges summarise relationships between attacker and target entities. All three are reachable from a single Streamlit dashboard that answers twelve decision-support queries and runs side-by-side backend comparisons, which is the view an evaluator actually cares about when judging how these models differ in practice.

[1] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization." In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. Funchal, Portugal: SCITEPRESS, Jan. 2018, pp. 108–116.

Beyond the core modelling work, the project covers a number of operational concerns. Data cleaning is non-trivial because the chosen dataset variant stores IP addresses as decimal integers and truncates timestamps to minute-second precision, so the ingestion pipeline has to reconstruct both. Pre-aggregated summary tables are defined alongside batch job SQL that refreshes them on a regular cadence during streaming rather than as a one-shot nightly job, which keeps the dashboard responsive while data is still flowing in. The same data path writes to all three backends in a single pass to avoid the consistency gaps that arise when one is synced from another after the fact. Everything runs from a single `docker compose up` on a laptop, which was a hard constraint rather than a nice-to-have given the single-author setting.

A number of things are explicitly out of scope. The deployment is single-node, so cluster configuration for Kafka, PostgreSQL, MongoDB, or Neo4j is not treated. There is no authentication layer because the academic context does not call for one. The dashboard is optimised for demonstration clarity rather than production user experience. Geolocation is simulated from IP hashes rather than fetched from a real GeoIP service, which is an assumption stated in Chapter 2.

1.3 System Overview

CyberSight DW runs as a ten-service Docker topology built around a Kafka-based streaming pipeline.[2] The producer service reads the CICIDS 2017 CSV files in chunks, applies an eight-step cleaning routine, and publishes each event as a JSON message to a Kafka topic named `network-events`. Producer and consumer start automatically with the rest of the stack, so the stream begins flowing as soon as the containers are up. The consumer service reads from that topic and, for every batch, writes to all three backends within the same iteration: PostgreSQL synchronously, MongoDB synchronously, and Neo4j via a background thread so that its slower graph writes do not stall the main loop. A separate ksqlDB service is initialised on startup with a set of tumbling-window tables defined over the same Kafka topic, which the dashboard reads for real-time panels. Figure 1.1 shows the full topology.

[2] Yasin Hessnawi. *CyberSight DW: Source Code Repository*. GitHub. https://github.com/yasinhessnawi1/CyberSight_DW. 2026.

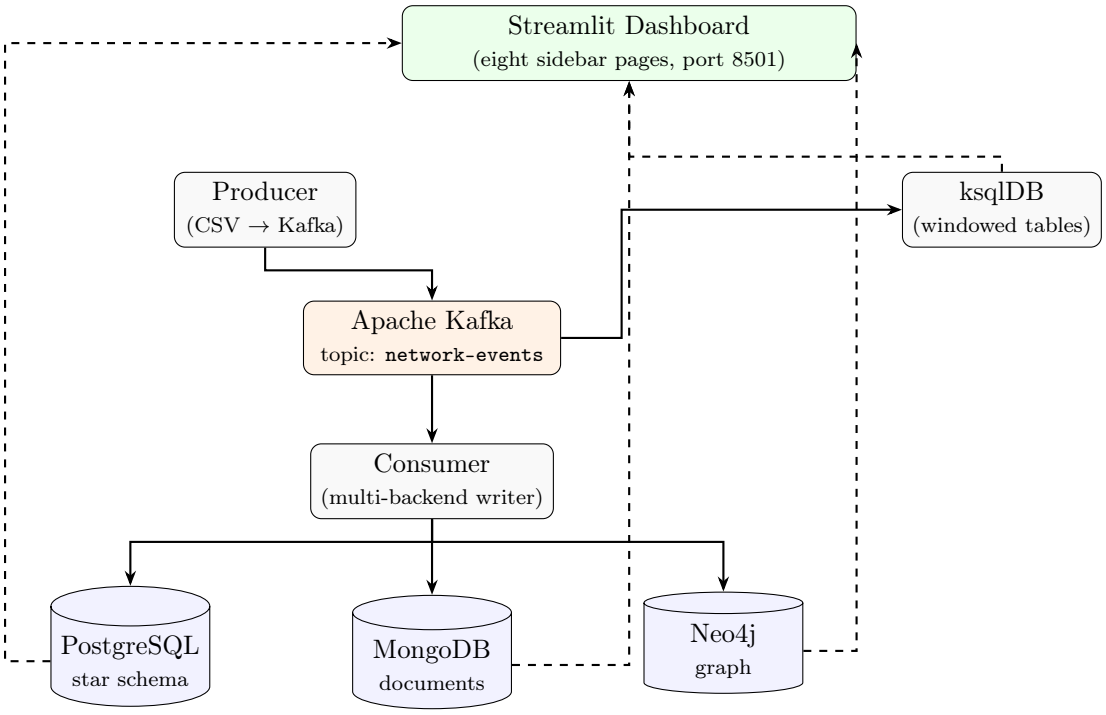

Figure 1.1: CyberSight DW system architecture. Solid arrows mark data flow along the streaming pipeline. Dashed arrows mark dashboard reads from the three backends and from the ksqlDB windowed tables.

The dashboard is analytical only. It does not control the pipeline; streaming runs autonomously once the stack starts, and the dashboard reads from PostgreSQL, MongoDB, Neo4j, and ksqlDB to render its eight pages. That separation keeps the comparison honest, because the backends all see the same input stream at the same time, and differences observed in the dashboard reflect differences in storage and query behaviour rather than differences in how the dashboard talks to each backend.

1.4 Users and Roles

Four types of users interact with CyberSight DW, and the dashboard is shaped around their habits. The **SOC analyst** is the primary user, someone watching ongoing threat activity throughout a shift. This role leans on the Overview and Threat Profiling pages to spot spikes and identify newly active attacking IPs, with occasional drill-downs into specific attack types during incident investigation. The analytical queries most relevant to this role are hourly attack volume, attack type breakdown, and top attacking sources.

The **threat intelligence researcher** is a secondary user studying longer patterns rather than live events. This role gravitates to the Time Analysis and Geo Analysis pages, reviewing weekly aggregations, protocol distributions, and country-level threat summaries. Export of summary data for external reports is part of the workflow, so the dashboard exposes table views rather than dashboards-only visualisations.

The **database administrator** is a technical user whose concern is the warehouse itself rather than its contents. This role checks Docker health, inspects ETL pipeline logs, reviews schema state, and monitors batch job execution. The administrator's work is mostly outside the dashboard, though the Backend Comparison page doubles as a health-and-status check because its query timings reveal when one backend is lagging.

The **academic evaluator** is the course instructor reviewing the system for correctness,

completeness, and depth. Unlike the other roles, the evaluator reads the report alongside the system, so the dashboard's Backend Comparison page is designed to demonstrate requirement 12 (the relational-versus-NoSQL comparison) in a single view that can be run during the demo.

1.5 Report Structure

The rest of this report builds up from context to detail and closes with comparative reflection. Chapter 2 covers background concepts and the assumptions the project rests on, and includes the glossary of acronyms for readers who want the terminology pinned down before the technical chapters begin. Chapter 3 describes the CICIDS 2017 dataset and the cleaning pipeline that turns raw CSV chunks into normalised event messages on Kafka. Chapter 4 is the densest chapter of the report: it lays out the full star schema with DDL, the six pre-aggregated summary tables, and the batch job SQL that populates them. Chapter 5 presents the two NoSQL implementations, MongoDB's embedded-document analogue of the star schema and Neo4j's aggregated property graph, with their own query examples. Chapter 6 covers the docker-compose topology and the Kafka streaming pipeline, and includes the ksqlDB extension. Chapter 7 describes the twelve decision-support queries, the Streamlit dashboard design, and two user scenarios that exercise the system end to end. Chapter 8 presents measured results from a full end-to-end run, covering data quality, backend row counts, and observed query performance. Chapter 9 compares the three backends across schema, query style, performance, and unique strengths, framing them as complementary rather than competing. Chapter 10 closes with reflections and limitations. Appendix A contains the full SQL listings referenced from Chapter 4, together with the dimension seed script, the ksqlDB initialisation, and the twelve decision-support query files.

Chapter 2

Background and Assumptions

2.1 Data Warehousing Fundamentals

A data warehouse and an operational database solve different problems even when they hold overlapping data. An operational database supports the day-to-day transactions of an application: rows are inserted, updated, and deleted, queries touch small numbers of records, and schema is shaped by transactional integrity. A data warehouse supports analytical work over consolidated history: rows accumulate rather than being overwritten, queries aggregate across millions of records, and schema is shaped by the questions the analyst is likely to ask. A warehouse does not replace the operational database; it sits downstream of it, fed by a pipeline that reshapes transactional data into a form that is cheap to aggregate.

The dimensional model is the standard way to shape that form. At its core are two kinds of table. A fact table holds one row per event of interest, with numeric measures (packet counts, byte counts, flow durations) and foreign keys into the dimension tables. A dimension table holds the descriptive context that surrounds an event, such as time, source, destination, attack type, and protocol in the case of this project. The join between a fact and its dimensions lets an analyst filter, group, and label events without dragging heavy context into the fact table itself. When the dimension tables join directly to the fact, the shape is a star schema; when dimensions further normalise into sub-dimensions, the shape is a snowflake. Stars are simpler and faster to query; snowflakes save storage at the cost of join depth. The CICIDS 2017 data fits a star cleanly, so CyberSight DW uses a star.

Fact tables grow quickly. A dataset of two million events produces a two-million-row fact table, and aggregating across the whole thing on every dashboard refresh is wasteful when the underlying question is often "how many attacks per hour." Pre-aggregated summary tables solve this. A summary table is computed from the fact and dimension tables by a batch job, and it holds exactly the grain needed by a common dashboard query: events per hour per attack category, for example, or attack counts per source IP per day. The dashboard queries the summary table, not the fact table, so response time is bounded by the summary's size rather than the fact's.

2.2 Streaming and Alternative Models

The relational model is not the only shape a warehouse can take, and for some questions it is not the most natural one. Document stores like MongoDB treat each record as a nested structure, so the context that a star schema would normalise into dimension tables can instead be embedded directly inside each event. Reading a single event then costs one lookup rather than one fact plus five dimension joins, at the price of storing repeated

context across millions of rows. For the cybersecurity events in this project, that trade is defensible: the dashboard often wants to render a single event's full context, and storage cost is secondary.

Graph databases like Neo4j go further in a different direction. Instead of normalising context into tables, they make the relationships between entities first-class. A source IP is a node, an attack type is a node, the edge that connects them says "this source used this attack," and a query that would be expensive in SQL (find all source IPs that share an attack type with a given source IP) becomes a natural two-hop traversal. The cost is that aggregation queries which scan every event are awkward in a graph, because graphs are not shaped for full-table scans.

Between ingestion and storage sits the streaming layer. Apache Kafka is a distributed commit log that decouples producers from consumers: a producer writes messages to a topic, and consumers read them at their own pace, with offsets that let a consumer resume where it left off after a restart. That decoupling is what makes it reasonable for one producer to feed three different backends at different speeds without one blocking the others. On top of Kafka, ksqlDB adds a streaming SQL engine that maintains materialised views over topics. These views refresh continuously as new events arrive, which is a different model from the batch summary tables in the relational warehouse. Summary tables answer "what has happened up to now over the full history"; ksqlDB tables answer "what has happened in the last minute, refreshed as the stream moves." Both are useful, and both appear in this project.

2.3 Assumptions

Every design rests on simplifications, and stating them openly is the only way a reader can judge where the conclusions hold and where they do not. The eleven assumptions in Table 2.1 fix the boundaries of the work. They cover the data, the infrastructure, and the user model, and every later chapter is consistent with them.

Table 2.1: Explicit assumptions the project rests on.

#	**Assumption**
A1	The CICIDS 2017 dataset is treated as production-like network telemetry rather than as toy data. The warehouse is shaped around real-world analytical questions, not around the particular quirks of a demonstration dataset.
A2	Geolocation is simulated. Source IP addresses in the dataset are synthetic, so country and region are derived from a deterministic hash of the IP rather than from a real GeoIP service. Geographic results reflect the simulation, not the real world.
A3	Timestamps are reconstructed. The chosen dataset variant stores timestamps in truncated `MM:SS.S` form, so full timestamps are synthesised from the filename (which identifies the day) and a uniform distribution across an 08:00–17:00 working window.
A4	Deployment is single-node. All services run on a single Docker host on a laptop. Cluster configuration for Kafka, PostgreSQL, MongoDB, and Neo4j is out of scope.

Continued on next page

Table 2.1 continued from previous page

#	**Assumption**
A5	Attack labels in the dataset are ground truth. The `Label` column is trusted as a correct threat classification for the purpose of this project, even though in a real system such labels would themselves be the output of a classifier and subject to error.
A6	Batch jobs are specified at the SQL level. The nightly batch job is specified as a SQL script rather than wired behind a scheduler, and the summary tables are populated from the fact and dimension tables as a separate job.
A7	A single Kafka topic carries all network events. The pipeline uses one topic (`network-events`) rather than partitioning by source, protocol, or attack category.
A8	Port numbers are not personally identifiable information. No anonymisation is applied to IP addresses or ports, which is consistent with how the source dataset is distributed.
A9	The three backends answer equivalent analytical questions using idioms native to each, but they are not full feature mirrors of one another. PostgreSQL and MongoDB store every raw event and aggregate at query time, while Neo4j stores only aggregated relationships and therefore cannot answer questions that require per-event detail. Cases where a backend cannot answer a query, or returns fewer columns than the others, are called out openly rather than hidden.
A10	The dashboard is optimised for clarity rather than for production ergonomics. Authentication, session management, and access control are out of scope.
A11	Each backend uses its native computation strategy rather than being constrained to match the others. PostgreSQL and MongoDB compute aggregates at query time over raw events; Neo4j pre-aggregates at ingestion and reads pre-computed counts from relationship properties. This asymmetry is deliberate, because the point of keeping three backends is to observe how different data models handle the same questions, not to equalise them into a single shape.

Chapter 3

Data and the ETL Pipeline

3.1 Dataset Description

The source data is CICIDS 2017, a labelled network intrusion detection dataset produced by the Canadian Institute for Cybersecurity. The dataset captures five consecutive weekdays of network traffic (Monday 3 July through Friday 7 July 2017) on a testbed that mixes benign user activity with a planned schedule of attacks. Each day's file contains between three hundred thousand and five hundred thousand flow records, and each record describes a single bidirectional network flow: source and destination addresses and ports, protocol, timestamp, packet and byte counts in each direction, derived rate measures, and a label identifying the flow as benign or as a specific attack type. Table 3.1 lists the five files, the attack types each introduces, and the exact record count of each.

Table 3.1: CICIDS 2017 source files used in this project.

File	**Attack types introduced**	**Records**
monday.csv	Benign traffic only, used as a baseline day	371,621
tuesday.csv	FTP-Patator and SSH-Patator brute force	322,078
wednesday.csv	DoS (Hulk, GoldenEye, Slowloris, Slowhttptest), Heartbleed	496,640
thursday.csv	Web attacks, Infiltration	362,075
friday.csv	DDoS, PortScan, Botnet	547,557
Total		**2,099,971**

The 2,099,971 records survive the cleaning process, since the `vishwa132` variant has already had its most problematic rows removed before publication. The distribution of attack types across days is uneven by design, since the dataset was constructed to study specific attack patterns in isolation. Monday holds only benign traffic, which serves as a behavioural baseline; Friday holds the largest volume and the most aggressive mix, with DDoS and PortScan together accounting for more than 250,000 attack events in a single day.

Several variants of CICIDS 2017 circulate online, and this project uses the `vishwa132/CICIDS-2017` variant hosted on Hugging Face.[1] Other options were evaluated first. The original UNB distribution site failed to resolve, which was consistent with it having been retired. A UNB mirror returned HTML landing pages rather than actual archives. A different Hugging Face variant, `ksay1o/CIC-IDS2017`, was missing critical columns: no source or destination IP, no timestamp, and no protocol, because it had been reduced to a machine-learning feature

[1] vishwa132. *CICIDS-2017 Dataset*. Hugging Face Datasets. https://huggingface.co/datasets/vishwa132/CICIDS-2017. 2024.

matrix rather than a full flow record. The `vishwa132` variant was the only freely accessible source that preserved every column the star schema needs.

The variant does have three quirks that the cleaning pipeline is built around. First, IP addresses are stored as unsigned 32-bit decimal integers rather than as dotted-quad strings, so every IP has to be converted before it can be stored in a PostgreSQL `INET` column or matched as a string in Neo4j. Second, timestamps are truncated to `MM:SS.S` format, losing the date and hour information entirely; full timestamps must be reconstructed from the source filename (which identifies the day) combined with a uniform distribution across an 08:00–17:00 working window. Third, attack labels are inconsistently punctuated across files: the same DoS variant may appear as `DoS Hulk`, `DoS-Hulk`, or `DoS-Hulk` with an en-dash or em-dash, so a straightforward equality comparison misses legitimate matches. The cleaning pipeline handles each of these explicitly.

3.2 Cleaning Pipeline

The producer applies an eight-step cleaning routine to every chunk of CSV data before sending it downstream. The steps are ordered so that each one takes the output of the previous as its input, and the order is deliberate. Column-name normalisation comes first because several later steps look for columns by name, and the Hugging Face variant's column names differ from the original UNB names in ways that break those lookups unless a rename map is applied. Decimal-to-dotted-quad IP conversion comes next because it depends on the normalised column names and because downstream steps treat the IP columns as strings.

The third step replaces positive and negative infinity with NaN. CICIDS 2017 is known to produce infinity values in its flow-rate columns (bytes per second and packets per second) when the duration of a flow rounds to zero, and leaving them in the data would break every numeric aggregation. The fourth step drops rows whose critical fields are missing after the first three steps, where "critical" means source IP, destination IP, label, and timestamp. Any row without those cannot produce a meaningful fact record. The fifth step fills remaining NaN values in numeric columns with zero, which is the sensible default for packet and byte counts.

The sixth step reconstructs timestamps. Since the source only provides `MM:SS.S`, the date component comes from the filename (Monday through Friday map to 2017-07-03 through 2017-07-07) and the hour-and-minute component is synthesised by distributing rows uniformly across the 08:00–17:00 working window. This synthesis preserves the temporal ordering within each file while producing timestamps usable by a warehouse. The seventh step decodes the protocol number into a protocol name (6 becomes TCP, 17 becomes UDP, 1 becomes ICMP, everything else becomes OTHER). The eighth step normalises the attack label through a multi-stage process that first normalises dash variants (en-dash and em-dash become a standard separator), then applies a case-insensitive lookup against a known-label map, then strips known suffixes such as `- Attempted`, and finally falls back to prefix-based categorisation. The same step also assigns an `attack_category` and a `severity` derived from the normalised label, and attaches a simulated source country and region via a deterministic hash of the IP against a sixteen-country lookup table.

Table 3.2 summarises the steps for reference.

Table 3.2: Cleaning steps applied by the producer before each chunk leaves for Kafka.

#	**Operation**	**Purpose**
1	Column-name normalisation	Rename `vishwa132` columns to match the names used downstream, so subsequent steps can find fields reliably.
2	Decimal IP to dotted quad	Convert 32-bit integer IPs to strings suitable for `INET` and graph-node keys.
3	Replace $\pm\infty$ with NaN	Flow-rate columns carry infinity when duration rounds to zero; these would break aggregations.
4	Drop rows missing critical fields	No meaningful fact can be built without source IP, destination IP, label, and timestamp.
5	Fill remaining NaN with 0	Packet and byte counts default to zero rather than NULL for numeric stability.
6	Reconstruct timestamps	Derive the date from the filename and spread rows across 08:00–17:00 to recover usable timestamps.
7	Decode protocol number	Map numeric protocol codes to names (TCP, UDP, ICMP, OTHER) through a shared mappings module.
8	Normalise label and enrich	Standardise attack labels across variants, assign category and severity, and attach simulated geolocation.

Two details of the cleaning strategy are worth calling out. The mappings used in steps 7 and 8 live in a shared module (`mappings.py`) that the producer, consumer, and both NoSQL writers all import, so the protocol-code-to-name and label-to-category truths are defined in exactly one place. This prevents the classic warehouse bug where two ingestion paths disagree on how to categorise the same raw value. The geolocation attached in step 8 is deliberately deterministic: the same IP produces the same country and region on every run, which keeps results reproducible across runs even though the geography itself is synthetic (see assumption A2 in Chapter 2).

3.3 Event Message Schema

After cleaning, every row is serialised to a JSON message with a fixed schema and pushed to the Kafka topic `network-events`. This message is the canonical event shape for the rest of the system: the consumer unpacks it into PostgreSQL dimension and fact rows, the MongoDB writer embeds it into a document, the Neo4j writer aggregates it into graph relationships, and ksqlDB reads it directly as a streaming input. Listing 3.1 shows the schema.

```
{
  "flow_id": "string",
  "source_ip": "10.0.0.5",
  "source_port": 443,
  "destination_ip": "10.0.0.10",
  "destination_port": 80,
  "protocol": "TCP",
  "timestamp": "2017-07-05T09:14:32Z",
  "flow_duration": 123456,
  "fwd_packets": 12,
  "bwd_packets": 8,
  "fwd_bytes": 5400,
  "bwd_bytes": 3200,
  "flow_bytes_per_sec": 43.88,
  "flow_packets_per_sec": 162.60,
  "label": "DoS Hulk",
  "attack_category": "DoS",
  "severity": "HIGH",
  "source_country": "Vietnam",
  "source_region": "Asia"
}
```

Listing 3.1: Canonical event message produced for each cleaned flow record.

The schema groups into four logical parts. Identification fields (`flow_id`, the two IP-and-port pairs, the protocol name, and the timestamp) name the participants and the time of the flow. Traffic measures (`flow_duration`, the four packet and byte counts, and the two rate columns) carry the numeric quantities the warehouse aggregates over. Classification fields (`label`, `attack_category`, `severity`) carry the enrichments that the cleaning pipeline attached. Geography fields (`source_country`, `source_region`) carry the simulated geolocation. Every field is either a string, a number, or a boolean, which keeps the message JSON-clean and makes it trivially readable by consumers written in any language. The absence of nested structures in the wire schema is intentional; MongoDB's embedding happens on the writer side, not on the wire.

3.4 ETL Flow and Loading

The full flow from source file to loaded backend has three stages: extraction and cleaning in the producer, transport through Kafka, and transformation and loading in the consumer. Figure 3.1 shows the shape of this pipeline.

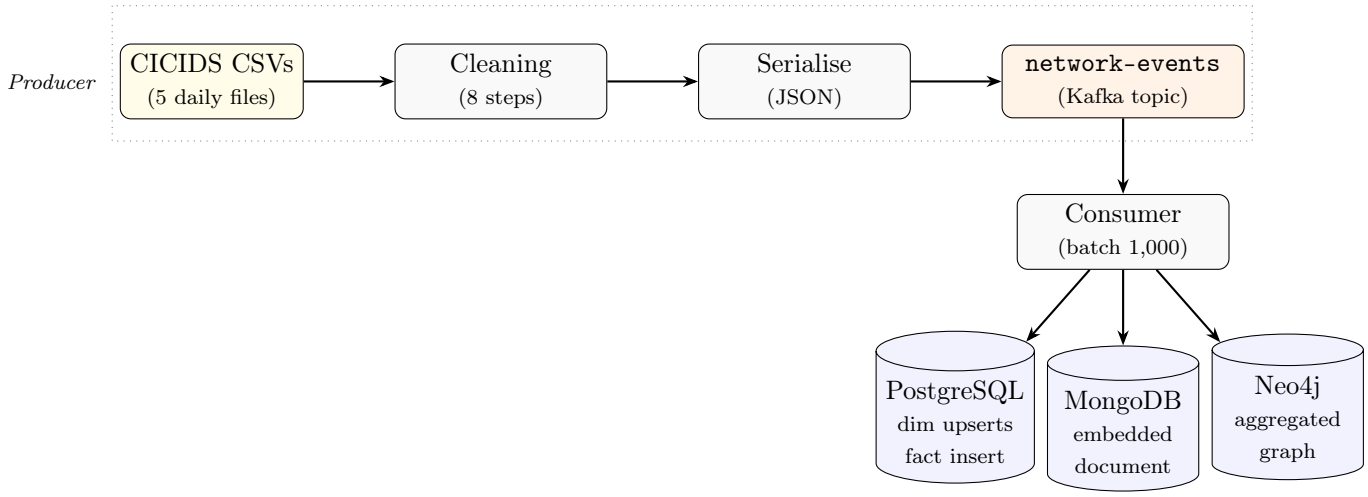

Figure 3.1: ETL flow. The producer reads CSV chunks, applies eight cleaning steps, serialises each row to JSON, and publishes to the Kafka topic. The consumer reads the topic in 1,000-message batches and writes every batch to all three backends in a single iteration.

The producer side reads the CICIDS files in chunks of 50,000 rows rather than as whole files, which keeps memory use bounded even for the largest Friday file. Each chunk is passed through `clean_dataframe()`, serialised row by row into JSON messages, and sent to the Kafka topic. At the end of all five files the producer sends a sentinel message (`{"_sentinel": "END_OF_STREAM"}`) that the consumer recognises and uses to trigger final aggregations.

The consumer side batches 1,000 messages at a time and writes each batch to PostgreSQL, MongoDB, and Neo4j in the same iteration. The PostgreSQL write is the most involved because it requires dimension-table upserts before the fact insert: for every event, the consumer upserts into `dim_time`, `dim_source`, `dim_destination`, `dim_attack`, and `dim_protocol`, retrieves the surrogate keys, and then inserts the fact row with those keys as foreign references. The MongoDB write transforms each event into an embedded document via `event_to_document()` and bulk-inserts the batch with `insert_many(ordered=False)`, which tolerates duplicate-key errors on individual documents without aborting the batch. The Neo4j write is enqueued to a background thread that maintains its own aggregation buffers and flushes them through `MERGE` statements. This asymmetric pattern, synchronous for the two faster backends and asynchronous for the slower one, is revisited in Chapter 6 where the trade-offs are examined directly. Kafka offsets are committed only after all three writes complete, which gives the pipeline at-least-once delivery semantics: a crash between write and commit causes the batch to be re-read on restart, and idempotent upserts ensure the replay is safe.

Chapter 4

Relational Data Warehouse Design

4.1 Schema Overview

The warehouse uses a classic star schema: five dimension tables surrounding a single fact table, with every dimension joining directly to the fact through a single-column foreign key. The design maps onto the natural decomposition of a network flow event. Who is attacking whom (`dim_source`, `dim_destination`), what kind of attack it is (`dim_attack`), when it happened (`dim_time`), and how it was carried (`dim_protocol`). The fact table, `fact_network_event`, holds one row per observed flow and carries seven numeric measures plus a boolean attack flag. Figure 4.1 shows the shape.

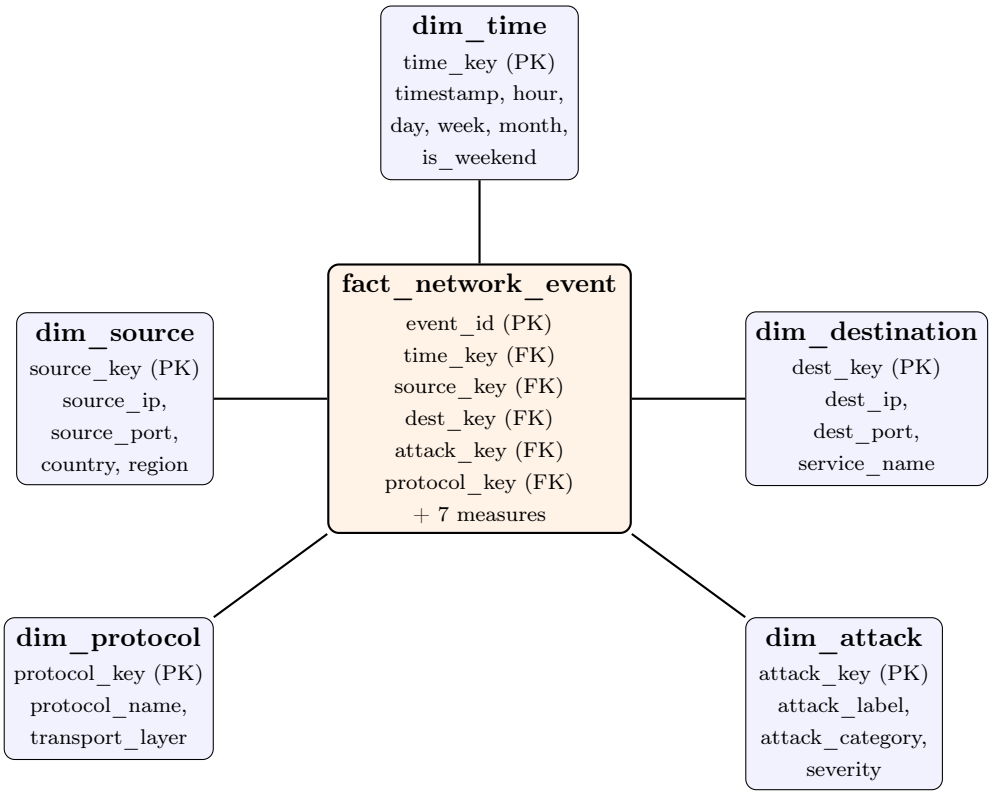

Figure 4.1: Star schema. Five dimension tables radiate from `fact_network_event`. Every dimension joins to the fact through a single surrogate key.

A snowflake variant was considered and rejected. Normalising geography into a separate `dim_country` table, or splitting attack category and severity out of `dim_attack`, would have added a second level of joins to queries that are meant to be cheap. At two million fact rows

and five dimensions, the storage savings from further normalisation are negligible while the query cost is noticeable. A single flat denormalised table was also rejected for the opposite reason: repeating attack category, severity, country, and service-name strings across every fact row would have bloated the table and made aggregations slower. The star sits at the right level of denormalisation for this workload.

4.2 Dimension Tables

Each dimension captures one axis of context. The designs are deliberately uneven because the dimensions have different cardinalities and different update patterns, and the granularity choice for each was made on its own merits rather than for uniformity.

dim_time

`dim_time` holds one row per distinct timestamp observed in the data rather than one row per calendar hour or day. This choice is driven by the dataset's temporal structure: after timestamp reconstruction (section 3.2), the data spans 2,705 distinct timestamps across five days. Per-timestamp rows let the fact table reference exact event times, and the derived columns (`hour`, `day_of_week`, `is_weekend`, `time_of_day`) let roll-up queries group at coarser grains without joining anything extra.

```sql
CREATE TABLE dim_time (
    time_key        SERIAL PRIMARY KEY,
    timestamp       TIMESTAMP NOT NULL,
    hour            SMALLINT NOT NULL CHECK (hour BETWEEN 0 AND 23),
    day             SMALLINT NOT NULL CHECK (day BETWEEN 1 AND 31),
    week            SMALLINT NOT NULL CHECK (week BETWEEN 1 AND 53),
    month           SMALLINT NOT NULL CHECK (month BETWEEN 1 AND 12),
    year            SMALLINT NOT NULL,
    day_of_week     VARCHAR(10) NOT NULL,
    is_weekend      BOOLEAN NOT NULL DEFAULT FALSE,
    time_of_day     VARCHAR(10) NOT NULL
);

CREATE UNIQUE INDEX idx_dim_time_ts ON dim_time(timestamp);
```

Listing 4.1: DDL for `dim_time`.

dim_source and dim_destination

The two endpoint dimensions share an identical structure: a surrogate primary key, an IP address stored as `INET`, a port number, and a set of descriptive attributes. The natural key is the `(ip, port)` pair, because the same IP can appear on many ports and each socket needs its own identity. Keeping the natural key as a `UNIQUE` constraint rather than as the primary key lets fact-table foreign keys remain compact four-byte integers instead of an `INET`-plus-`INTEGER` composite. Source rows carry country and region (populated from the simulated geolocation described in assumption A2 of Chapter 2), and destination rows carry service name and service type (populated from a port-to-service mapping module).

```sql
CREATE TABLE dim_source (
    source_key      SERIAL PRIMARY KEY,
    source_ip       INET NOT NULL,
    source_port     INTEGER NOT NULL CHECK (source_port BETWEEN 0 AND 65535),
    country         VARCHAR(100) DEFAULT 'Unknown',
    region          VARCHAR(100) DEFAULT 'Unknown',
    ip_class        VARCHAR(10),
```

```sql
    UNIQUE (source_ip, source_port)
);

CREATE INDEX idx_dim_source_ip ON dim_source(source_ip);
```

Listing 4.2: DDL for `dim_source`.

```sql
CREATE TABLE dim_destination (
    dest_key        SERIAL PRIMARY KEY,
    dest_ip         INET NOT NULL,
    dest_port       INTEGER NOT NULL CHECK (dest_port BETWEEN 0 AND 65535),
    service_name    VARCHAR(50) DEFAULT 'Unknown',
    service_type    VARCHAR(50) DEFAULT 'Unknown',
    UNIQUE (dest_ip, dest_port)
);

CREATE INDEX idx_dim_dest_ip ON dim_destination(dest_ip);
```

Listing 4.3: DDL for `dim_destination`.

A full end-to-end run produces 411,658 distinct source sockets and 39,928 distinct destination sockets, which gives a sense of the cardinality difference between the two ends of a flow. Many attacks originate from many ephemeral source ports against a small set of service ports on the target side.

dim_attack

`dim_attack` stores one row per distinct attack label rather than one row per category. The label granularity is useful because the same category contains meaningfully different attack patterns: `DoS Hulk` and `DoS Slowloris` are both in the `DoS` category but differ in technique, flow duration, and signature. The `attack_category` column provides the roll-up path for queries that only need the coarse grouping. A `severity` rating on a four-level ordinal scale (`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`) is added as an analytical dimension not present in the raw data, and is populated by the cleaning pipeline via the shared mappings module.

```sql
CREATE TABLE dim_attack (
    attack_key      SERIAL PRIMARY KEY,
    attack_label    VARCHAR(100) NOT NULL UNIQUE,
    attack_category VARCHAR(50)  NOT NULL,
    severity        VARCHAR(10)  NOT NULL
                    CHECK (severity IN ('LOW','MEDIUM','HIGH','CRITICAL')),
    description     TEXT
);
```

Listing 4.4: DDL for `dim_attack`.

dim_protocol

`dim_protocol` is the smallest dimension, holding one row per known protocol name. Unlike the other dimensions, which are populated dynamically as events arrive, this one is seeded at schema-init time with the closed set `{TCP, UDP, ICMP, OTHER}`. The seed approach sidesteps race conditions during parallel consumer inserts and ensures a consistent reference set for summary tables.

```sql
CREATE TABLE dim_protocol (
    protocol_key    SERIAL PRIMARY KEY,
    protocol_name   VARCHAR(20) NOT NULL UNIQUE,
    transport_layer VARCHAR(20) NOT NULL
);
```

Listing 4.5: DDL for `dim_protocol`.

4.3 Fact Table

`fact_network_event` is the centre of the schema. It holds one row per cleaned flow event, five foreign keys into the dimensions, seven numeric measures, and a boolean flag that makes attack filtering cheap (`is_attack`). The flag is redundant with the attack-category roll-up in `dim_attack`, but materialising it on the fact table lets queries filter on attacks without joining the attack dimension, which matters for the very common "how many attack events" question.

```sql
CREATE TABLE fact_network_event (
    event_id            BIGSERIAL PRIMARY KEY,
    time_key            INTEGER NOT NULL REFERENCES dim_time(time_key),
    source_key          INTEGER NOT NULL REFERENCES dim_source(source_key),
    dest_key            INTEGER NOT NULL REFERENCES dim_destination(dest_key),
    attack_key          INTEGER NOT NULL REFERENCES dim_attack(attack_key),
    protocol_key        INTEGER NOT NULL REFERENCES dim_protocol(protocol_key),
    flow_duration       BIGINT  NOT NULL DEFAULT 0,
    fwd_packets         INTEGER NOT NULL DEFAULT 0,
    bwd_packets         INTEGER NOT NULL DEFAULT 0,
    fwd_bytes           BIGINT  NOT NULL DEFAULT 0,
    bwd_bytes           BIGINT  NOT NULL DEFAULT 0,
    flow_bytes_per_sec  NUMERIC(15,4) DEFAULT 0,
    flow_pkts_per_sec   NUMERIC(15,4) DEFAULT 0,
    is_attack           BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE INDEX idx_fact_time           ON fact_network_event(time_key);
CREATE INDEX idx_fact_source         ON fact_network_event(source_key);
CREATE INDEX idx_fact_attack         ON fact_network_event(attack_key);
CREATE INDEX idx_fact_is_attack      ON fact_network_event(is_attack);
CREATE INDEX idx_fact_time_attack    ON fact_network_event(time_key, attack_key);
CREATE INDEX idx_fact_protocol       ON fact_network_event(protocol_key);
CREATE INDEX idx_fact_dest           ON fact_network_event(dest_key);
CREATE INDEX idx_fact_is_attack_time ON fact_network_event(is_attack, time_key);
```

Listing 4.6: DDL for `fact_network_event`.

The index selection reflects the query patterns the dashboard actually exercises. Single-column indexes on each foreign key support simple filter-and-count queries. The `(time_key, attack_key)` composite index supports the hourly-attack timeline, which is the most frequently run dashboard query. The `(is_attack, time_key)` composite index supports attack-only time-range filters without scanning the full fact table. All eight indexes are created up front because they are cheap to maintain relative to the gains on read, and the fact table is append-only so there is no index-churn cost from updates.

4.4 Loading the Star Schema

Loading the star is more involved than loading a flat table, because the fact row needs surrogate keys that only exist once the corresponding dimension rows exist. The consumer follows the classic pattern: for every event, upsert into each dimension, retrieve the surrogate key that was either inserted or matched, and then insert the fact row with those keys. Listing 4.7 shows the shape of a dimension upsert. The example is for `dim_source`, but the same idiom applies to the other four dimensions.

```sql
INSERT INTO dim_source (source_ip, source_port, country, region, ip_class)
VALUES (%s, %s, %s, %s, %s)
ON CONFLICT (source_ip, source_port)
DO UPDATE SET country = EXCLUDED.country
RETURNING source_key;
```

Listing 4.7: Dimension upsert pattern. An insert is attempted; on conflict with the natural-key constraint, the existing row's primary key is returned instead.

The `DO UPDATE` clause is there to force `RETURNING` to fire on both the insert path and the conflict path. A pure `DO NOTHING` conflict handler would skip the `RETURNING` on collisions, leaving the code without a key to use for the fact insert. Updating the country field to itself is harmless and keeps the pattern uniform. Once all five dimension keys are in hand, the fact insert is straightforward:

```sql
INSERT INTO fact_network_event (
    time_key, source_key, dest_key, attack_key, protocol_key,
    flow_duration, fwd_packets, bwd_packets, fwd_bytes, bwd_bytes,
    flow_bytes_per_sec, flow_pkts_per_sec, is_attack
) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s);
```

Listing 4.8: Fact insert using the keys returned from the five dimension upserts.

The consumer batches 1,000 events per iteration, which bounds the round-trips to PostgreSQL and keeps transactional behaviour predictable. The at-least-once delivery semantics and their replay tolerance were covered in section 3.4; the only relational-side implication is that dimension upserts absorb replays cleanly (insert-or-match), while fact rows may duplicate under rare restart conditions. This is tolerated deliberately, because dashboard queries aggregate rather than count exact rows.

4.5 Pre-Aggregated Summary Tables

A two-million-row fact table is small by warehouse standards but still too large to scan on every dashboard refresh. Six summary tables pre-compute the aggregates the dashboard queries most often, at grains chosen to match what the UI actually asks for. Table 4.1 lists them.

Table 4.1: Pre-aggregated summary tables, their grain, and the queries they back.

Table	Grain (one row per)	Primary queries served
summary_attack_hourly	hour, attack label	Q1, Q2, Q7, Q12
summary_top_sources	day, source IP	Q3, Q9
summary_protocol_daily	day, protocol	Q5
summary_severity_daily	day, severity	Q10
summary_dest_port_daily	day, port, attack category	Q6
summary_weekend_weekday	weekend flag, attack category	Q11

Each summary table has a composite unique constraint on its grain columns, which is what makes the batch jobs idempotent. Re-running a batch job on the same fact data produces the same output rows, and an `ON CONFLICT DO UPDATE` clause refreshes the aggregated measures rather than inserting duplicates. This matters because the pipeline runs the batch jobs repeatedly during streaming, not once at the end.

The most-used summary, `summary_attack_hourly`, holds 250 rows after a full load, which is four orders of magnitude smaller than the two-million-row fact. The largest, `summary_top_sources`, holds 290 rows. Together the six summaries keep the dashboard's hottest queries sub-second even on a laptop, which is what makes the live-streaming experience watchable rather than sluggish. A derived column worth noting is the `threat_score` on `summary_top_sources`, which combines attack share, distinct target count, and a severity weight into a single ranking heuristic. It is a soft measure rather than a hard one, but it is enough to surface the most active attackers at the top of the dashboard's threat list without requiring the analyst to sort on three columns at once.

4.6 Batch Job SQL

The batch jobs are plain SQL scripts, one per summary table, executed by the consumer at regular intervals during streaming and once more at end-of-stream. Each script follows the same three-part shape: an `INSERT INTO` naming the target summary, a `SELECT` that computes the aggregate from the fact and dimensions, and an `ON CONFLICT ... DO UPDATE` clause that refreshes the measures on re-run. Listing 4.9 shows the hourly script in full, and Listing 4.10 shows the top-sources script. The remaining four scripts (protocol daily, severity daily, destination port, weekend vs weekday) follow the same pattern and are reproduced in Appendix A.

```sql
-- Batch Job: Populate summary_attack_hourly
-- Run after each load of fact_network_event

INSERT INTO summary_attack_hourly
    (hour_bucket, attack_category, attack_label, event_count,
     total_bytes, avg_duration_ms, unique_sources)
SELECT
    date_trunc('hour', dt.timestamp)      AS hour_bucket,
    da.attack_category,
    da.attack_label,
    COUNT(*)                              AS event_count,
    SUM(f.fwd_bytes + f.bwd_bytes)        AS total_bytes,
    AVG(f.flow_duration / 1000.0)         AS avg_duration_ms,
    COUNT(DISTINCT f.source_key)          AS unique_sources
FROM fact_network_event f
JOIN dim_time   dt ON f.time_key   = dt.time_key
JOIN dim_attack da ON f.attack_key = da.attack_key
GROUP BY date_trunc('hour', dt.timestamp), da.attack_category, da.attack_label
ON CONFLICT (hour_bucket, attack_label)
DO UPDATE SET
    event_count     = EXCLUDED.event_count,
    total_bytes     = EXCLUDED.total_bytes,
    avg_duration_ms = EXCLUDED.avg_duration_ms,
    unique_sources  = EXCLUDED.unique_sources,
    created_at      = NOW();
```

Listing 4.9: `batch_hourly.sql`: populates `summary_attack_hourly`.

The hourly script joins the fact table to `dim_time` and `dim_attack`, buckets the timestamps to the hour with `date_trunc`, and aggregates four measures: event count, total bytes (summed across both directions of each flow), average duration in milliseconds, and the count of distinct source surrogate keys. The grouping is by hour, attack category, and attack label, which lets a single row in the summary answer both category-level and label-level queries without recomputation.

```sql
-- Batch Job: Populate summary_top_sources
-- Run after each load of fact_network_event

```

```sql
INSERT INTO summary_top_sources
    (day_bucket, source_ip, country, attack_count,
     distinct_targets, primary_attack, threat_score)
SELECT
    date_trunc('day', dt.timestamp)::DATE            AS day_bucket,
    ds.source_ip,
    ds.country,
    COUNT(*) FILTER (WHERE f.is_attack = TRUE)       AS attack_count,
    COUNT(DISTINCT f.dest_key)                       AS distinct_targets,
    MODE() WITHIN GROUP (ORDER BY da.attack_label)   AS primary_attack,
    ROUND(
        (COUNT(*) FILTER (WHERE f.is_attack = TRUE)::NUMERIC
         / NULLIF(COUNT(*), 0))
        * COUNT(DISTINCT f.dest_key)
        * CASE MAX(da.severity)
            WHEN 'CRITICAL' THEN 4
            WHEN 'HIGH'     THEN 3
            WHEN 'MEDIUM'   THEN 2
            ELSE 1
          END
    , 2)                                              AS threat_score
FROM fact_network_event f
JOIN dim_time   dt ON f.time_key   = dt.time_key
JOIN dim_source ds ON f.source_key = ds.source_key
JOIN dim_attack da ON f.attack_key = da.attack_key
GROUP BY date_trunc('day', dt.timestamp)::DATE, ds.source_ip, ds.country
ON CONFLICT (day_bucket, source_ip)
DO UPDATE SET
    attack_count     = EXCLUDED.attack_count,
    distinct_targets = EXCLUDED.distinct_targets,
    primary_attack   = EXCLUDED.primary_attack,
    threat_score     = EXCLUDED.threat_score,
    created_at       = NOW();
```

Listing 4.10: `batch_sources.sql`: populates `summary_top_sources`, including the `threat_score` heuristic.

The top-sources script is the more interesting of the two because it derives a composite threat score rather than a simple aggregate. The `FILTER (WHERE f.is_attack = TRUE)` clause counts only attack events. The `MODE() WITHIN GROUP` picks the most common attack label for the source as a summary label, which gives the dashboard something readable rather than a raw attack-category histogram. The `threat_score` formula combines attack share (attack events over total events), distinct-target count, and a severity weight derived from the worst severity observed for that source on that day. The score is rounded to two decimal places and stored as `NUMERIC(12,2)`, a width that was widened from the original `NUMERIC(5,2)` after a numeric-overflow error surfaced at full dataset scale. That widening is a good reminder that warehouse DDL evolves with the data it ingests, and that precision choices made at design time sometimes need revisiting once real volumes are seen.

The cadence of batch job execution matters. Rather than running once at end-of-stream, the consumer runs all six scripts every 10,000 events during streaming. This keeps the summary tables consistently fresh during a long ingestion run, which is what makes the dashboard's live-streaming view useful. The final run at end-of-stream ensures the last partial batch is reflected in the summaries as well. The choice to refresh incrementally rather than nightly-only is a deliberate deviation from the textbook batch pattern, and it is possible precisely because the summary tables are small enough to re-compute cheaply.

Chapter 5

NoSQL Implementations

5.1 Approach and Structure

The two NoSQL backends hold the same cleaned events as the relational warehouse but in very different shapes. MongoDB folds star-schema context into each document through embedding, turning a single-event read into one collection lookup. Neo4j goes further: it discards event-level detail almost entirely and keeps only aggregated relationships between entities, making graph traversal the primary access pattern. Both choices are idiomatic for their respective systems, and both sub-chapters below follow a parallel structure (design philosophy, concrete schema, indexing or constraints, write path, and summary-table analogue) so the similarities and differences stay visible.

5.2 MongoDB Implementation

The MongoDB design takes the opposite stance of the star schema. Where the relational warehouse splits an event across a fact row and five dimension rows, the document model stores a single nested document with the entire star-schema context inlined. A typical dashboard query that would join the fact to `dim_time`, `dim_source`, and `dim_attack` in PostgreSQL becomes a single-collection lookup in MongoDB, at the price of storing repeated attack category, severity, country, and service-name strings across millions of documents. At this project's scale, that trade is clearly worth it. Storage is cheap, dashboard latency is not, and the use case is read-heavy analytical work rather than transactional updates that would need normalisation for consistency.

The `network_events` collection is the primary store. Every cleaned event becomes one document with six nested sub-documents that mirror the five dimensions and the seven measures of the relational model. Listing 5.1 shows the shape produced by `event_to_document()`. The timestamp is kept at the top level as a native BSON date for range queries, and the `time` sub-document duplicates it into rolled-up components (hour, day-of-week, is-weekend, time-of-day) to avoid recomputing those on every query.

```
{
  "_id": ObjectId("..."),
  "timestamp": ISODate("2017-07-05T09:14:32Z"),
  "time": {
    "hour": 9, "day": 5, "week": 27, "month": 7, "year": 2017,
    "day_of_week": "Wednesday",
    "is_weekend": false,
    "time_of_day": "Morning"
  },
  "source": {
    "ip": "192.168.10.5", "port": 54321,
```

```
    "country": "Vietnam", "region": "Asia"
  },
  "destination": {
    "ip": "192.168.10.50", "port": 80,
    "service_name": "HTTP", "service_type": "Web"
  },
  "attack": {
    "label": "DoS Hulk", "category": "DoS",
    "severity": "HIGH", "is_attack": true
  },
  "protocol": {
    "name": "TCP", "transport_layer": "Transport"
  },
  "metrics": {
    "flow_duration": 123456,
    "fwd_packets": 12, "bwd_packets": 8,
    "fwd_bytes": 5400, "bwd_bytes": 3200,
    "flow_bytes_per_sec": 43.88,
    "flow_pkts_per_sec": 162.60
  }
}
```

Listing 5.1: MongoDB document produced by `event_to_document()` for one cleaned flow event.

The boolean `attack.is_attack` is a small but deliberate addition. It is derived from `attack.category` (true whenever the category is not `Normal`), and materialising it lets queries filter on attacks with a boolean comparison instead of a string check, which is cheaper and matches MongoDB's indexing model better. The pattern echoes the `is_attack` column on `fact_network_event`, and exists for the same reason.

Five indexes are created on `network_events`, chosen to match the filter patterns the dashboard issues most often: single-field indexes on `timestamp`, `attack.category`, `attack.is_attack`, and `source.ip`, plus a compound index on `(attack.category, timestamp)` for the combined attack-timeline query. The strategy mirrors the fact table's index selection (section 4.3), adapted to dotted-path field names.

The write path during streaming is intentionally simple. The consumer hands each batch of raw events to `MongoWriter.write_batch()`, which transforms every event into a document and calls `insert_many(docs, ordered=False)`. The `ordered=False` flag lets the insert proceed past per-document errors (duplicate key collisions, schema violations on individual fields) rather than aborting the entire batch on the first problem, which is important for streaming workloads where a few bad rows should not halt the pipeline.

The MongoDB analogue of the PostgreSQL summary tables is the `attack_hourly_summary` collection, which mirrors `summary_attack_hourly` from the relational side. Unlike the relational case, which recomputes summaries from scratch with `INSERT ... ON CONFLICT DO UPDATE`, the MongoDB summary is built incrementally. Inside `MongoWriter.write_batch()`, after the main insert, the events are aggregated in Python by `(hour_bucket, attack_label)` and then flushed to MongoDB using bulk `UpdateOne` operations with `upsert=True` and `$inc` operators. Listing 5.2 shows the shape.

```
UpdateOne(
  filter = { hour_bucket: ..., attack_label: ... },
  update = {
    $inc: { event_count: N, total_bytes: B },
    $set: { attack_category: cat, severity: sev },
    $setOnInsert: { hour_bucket: ..., attack_label: ... }
  },
  upsert = true
)
```

Listing 5.2: Incremental summary update for `attack_hourly_summary`. Each `UpdateOne` upserts a row and increments counters rather than recomputing them.

The incremental pattern has two consequences worth noting. The summary stays fresh on every batch without needing a separate batch-job script, and the counters accumulate correctly across restarts because the operation is idempotent at the row level (though not at the event level, which matters under replay). A unique compound index on `(hour_bucket, attack_label)` backs the upsert target and guarantees that the bulk writes do not create duplicate summary rows under concurrency.

5.3 Neo4j Implementation

The Neo4j design makes a stronger claim than MongoDB does. Where MongoDB keeps every event and reshapes only the context, Neo4j discards the events themselves and keeps only aggregated relationships between entities. A single `ATTACKED` edge from one source IP to one destination IP represents not one flow but every flow between those two addresses, with properties carrying the count, total bytes, first-seen, and last-seen timestamps. The rationale is operational. Storing five hundred thousand attack events as individual edges would make graph traversal slow and expensive: a query that wants to know "which IPs does this source attack most" would need to aggregate across half a million edges, which is the opposite of what graph databases are for. Aggregating at write time keeps the graph small enough to traverse efficiently and lets the structure of the graph carry the answers directly.

The model uses five node types and six relationship types, shown in Figure 5.1. Each node type corresponds to a dimension-like concept, though the mapping is not one-to-one with the star schema. `SourceIP` and `DestinationIP` split what `dim_source` and `dim_destination` separate on the relational side. `AttackType` corresponds to a row of `dim_attack`. `Protocol` corresponds to a row of `dim_protocol`. `TimeWindow` is the one deliberate simplification: it represents an hourly bucket rather than a per-timestamp row, because hourly is the finest grain at which the dashboard's graph-oriented queries actually ask questions.

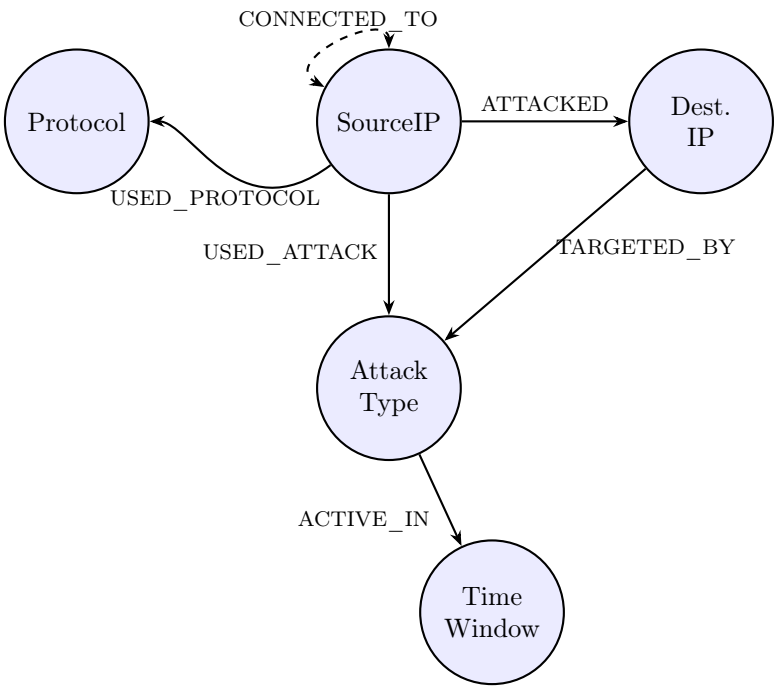

Figure 5.1: Neo4j graph model. Five node types connected by six relationship types. Solid arrows are written incrementally during streaming; the dashed `CONNECTED_TO` is derived in a post-load step.

The relationships carry the analytical load. `ATTACKED` holds the source-to-destination attack

flow, aggregated by endpoint pair, with properties `count`, `total_bytes`, `first_seen`, and `last_seen`. `USED_ATTACK` connects a source IP to an attack type it has used, with a count and timestamps. `TARGETED_BY` connects a destination IP to the attack types that hit it, which supports the "what is this target's threat profile" question. `USED_PROTOCOL` connects a source IP to a protocol, which supports protocol-level attacker profiling. `ACTIVE_IN` connects an attack type to the hourly time windows it was observed in, which gives the graph a time dimension for queries that need one. The sixth relationship, `CONNECTED_TO`, is different and deserves its own mention at the end of this section.

Uniqueness constraints are declared up front on every node type, one per natural key. `SourceIP.ip`, `DestinationIP.ip`, `AttackType.label`, `Protocol.name`, and `TimeWindow.hour_bucket` are all constrained unique, which is what makes the `MERGE` pattern safe. `MERGE` looks for a pattern and creates it if it does not exist, and without uniqueness constraints a concurrent write could create two nodes for the same IP, destroying the graph's integrity. Declaring the constraints at schema-init time lets `MERGE` run with confidence.

Writing happens in aggregated batches rather than event-by-event. The consumer hands a batch of raw events to `Neo4jWriter.write_batch()`, which aggregates them in Python using `defaultdicts` keyed by the endpoint tuples that identify each relationship type. The aggregation produces five parameter lists (one for each streaming relationship type), and each list is written to Neo4j using the `UNWIND $batch AS row ... MERGE ... ON CREATE SET ... ON MATCH SET` idiom. The `ON CREATE SET` branch sets initial property values, and the `ON MATCH SET` branch increments the counters and updates the `last_seen` timestamp. Listing 5.3 shows the pattern for the `ATTACKED` edge, which is the most loaded relationship in the graph.

```
UNWIND $batch AS row
MERGE (s:SourceIP {ip: row.src_ip})
  ON CREATE SET s.country = row.country, s.region = row.region
MERGE (d:DestinationIP {ip: row.dst_ip})
  ON CREATE SET d.service_name = row.service_name,
                d.service_type = row.service_type
MERGE (s)-[r:ATTACKED]->(d)
  ON CREATE SET r.count       = row.cnt,
                r.total_bytes = row.total_bytes,
                r.first_seen  = row.first_seen,
                r.last_seen   = row.last_seen
  ON MATCH  SET r.count       = r.count + row.cnt,
                r.total_bytes = r.total_bytes + row.total_bytes,
                r.last_seen   = row.last_seen
```

Listing 5.3: Aggregated write for the `ATTACKED` relationship. The `UNWIND + MERGE + ON CREATE/ON MATCH` idiom is idempotent and aggregates counts across batches.

Because the relationships themselves carry the aggregated counts, Neo4j does not need a separate summary-table analogue. The relationships *are* the summary. A query asking "how many events per attack type per hour" in PostgreSQL reads from `summary_attack_hourly`; the same question in Neo4j reads the `event_count` property directly off `ACTIVE_IN` edges. This is the tightest expression in the project of the idea introduced in Chapter 2: the graph pushes aggregation to write time, while the relational warehouse pushes it to periodic batch jobs. The trade is that the graph cannot answer questions that require per-event detail (such as average flow duration across raw flows), and it returns fewer columns than the other two backends for queries that ask for such detail.

The `CONNECTED_TO` relationship is the one piece of the graph that is not written incrementally. It is derived after the end-of-stream sentinel fires, by running a single Cypher query that finds all pairs of source IPs sharing at least one attack type and materialises a symmetric edge between them with the list of shared attacks and the count. The query walks `(s1)-[:USED_ATTACK]->(a)<-[:USED_ATTACK]-(s2)` and uses an ordering predicate

(`s1.ip < s2.ip`) to write each pair only once. This is the co-attacker graph that enables the dashboard's Q8 query in Chapter 7, and it is the clearest example in the project of a question that is natural in a graph and awkward in either of the other two backends.

After a full end-to-end run, the graph contains 19,303 nodes across the five types and 96,901 relationships across the six types. The `ATTACKED` relationship alone aggregates 517,000 attack events into 62,107 edges, a reduction factor of roughly 8:1 that captures how much repetition exists in the underlying attack patterns.

5.4 Comparing the Two NoSQL Shapes

Table 5.1 places the two NoSQL models side by side on the dimensions that differ most.

Table 5.1: MongoDB and Neo4j represent the same data through opposite philosophies.

Dimension	**MongoDB**	**Neo4j**
Event retention	Every event is stored as a document	Events are discarded; only aggregated edges remain
Schema shape	Nested document with embedded context	Labelled nodes connected by typed relationships
Aggregation time	Mostly at query time (`$group`)	Mostly at write time (counts on edges)
Natural question	"Show me this event and everything about it"	"How are these entities connected to each other"
Summary analogue	Separate collection maintained by incremental upserts	Implicit in the relationship properties themselves
Weakness	Full-collection scans for broad aggregations	No answer for per-event measures (e.g. avg duration)

The practical consequence is that the two NoSQL backends are complementary rather than substitutable. For questions centred on a single event or a narrow time slice with rich context, MongoDB answers directly from one document. For questions about how entities relate to each other across the whole dataset, Neo4j answers from the structure of the graph itself. A workload dominated by either kind of question would lean on one backend; a workload that mixes them benefits from both. The PostgreSQL star schema sits between these two extremes, with aggregation at query time like MongoDB but with the normalised structure that makes multi-dimension joins fast. Chapter 9 picks up this three-way positioning in full.

Chapter 6

Infrastructure and Streaming

6.1 Docker Topology

The entire stack runs from a single `docker-compose.yml` file. Ten services are defined, each pinned to a specific image version so that the composition is reproducible: two coordination services (Zookeeper, Kafka), one streaming-SQL service (ksqlDB server), three databases (PostgreSQL, MongoDB, Neo4j), two ETL services (producer, consumer), a one-shot initialiser for the ksqlDB tables, and the dashboard. A tenth service, `api`, is a small FastAPI instance that exposes two diagnostic endpoints: a log-tailing endpoint that merges log files from all services into one feed, and a dead-letter-queue browser that reads from the `network-events-dlq` Kafka topic. It also exposes a `/health` endpoint. Figure 6.1 shows the dependency graph.

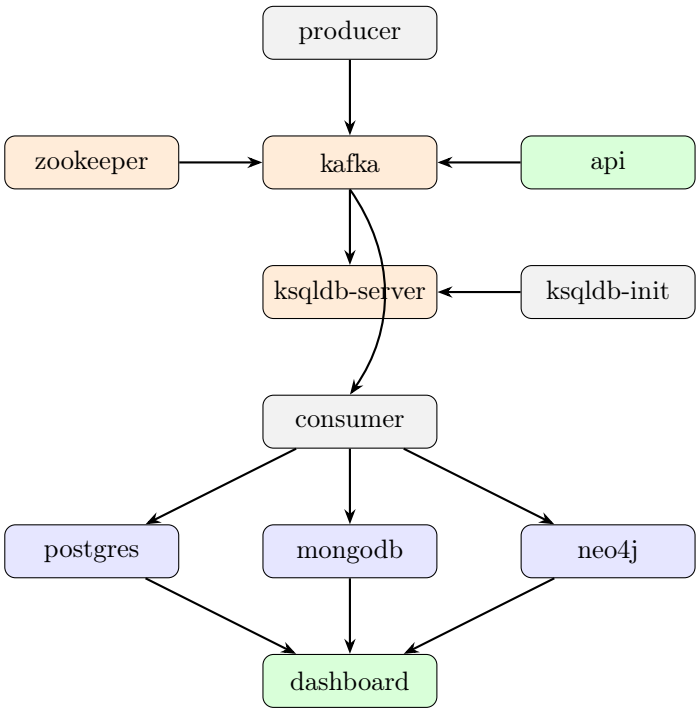

Figure 6.1: Data flow across the Docker services. The producer streams cleaned events into kafka; the consumer reads from kafka and writes to all three databases; the dashboard reads from the databases. The api service publishes diagnostic messages into kafka alongside the main stream, and ksqldb-server materialises tumbling-window views over the same topic. Orange nodes are infrastructure, blue nodes are databases, grey nodes are ETL services, and green nodes are the application layer.

The dependency pattern uses Compose's `depends_on` with `condition: service_healthy`, which means a service does not start until every service it depends on reports as healthy, not merely as running. This matters because containers accept TCP connections well before their underlying service is actually ready: PostgreSQL's port opens seconds before `pg_isready` returns success, and Kafka's port opens before the broker has registered itself with Zookeeper. The health checks are defined per database: `pg_isready -U postgres` for PostgreSQL, `mongosh` ping for MongoDB, an HTTP probe on port 7474 for Neo4j, the broker API check for Kafka, and a curl to `/info` for ksqlDB. The consumer is the service most affected by this: it depends on all four stateful services being healthy, which prevents it from starting to poll Kafka before its write targets exist.

Two infrastructure choices are worth noting. First, the Confluent images (Kafka, Zookeeper, ksqlDB) are pinned to version 7.6.0 rather than a floating tag. Version 7.6.0 ships native ARM64 builds, which avoids Rosetta emulation on Apple Silicon. Earlier versions required the `platform: linux/amd64` override, which ran through emulation and suffered out-of-memory kills on the JVM-based services. Second, the `etl_data` volume that holds the CICIDS CSV files is declared as an external named volume rather than a bind mount. On macOS, Docker Desktop uses `virtiofs` for bind mounts between host and container, and reading hundred-megabyte CSV files through `virtiofs` triggered `EDEADLK` errors from pandas. Using a named volume backed by the Linux VM's native filesystem bypasses `virtiofs` entirely and fixed the issue without code changes.

Port mappings follow a simple convention: each database exposes its native internal port to a non-standard host port to avoid collisions with any local installation. PostgreSQL is on host port 5433 (internal 5432), MongoDB on 27018 (internal 27017), and Neo4j's Bolt on 7688 (internal 7687). The dashboard stays on its conventional 8501, Kafka on 9092, and ksqlDB's REST on 8088.

6.2 Streaming Pipeline

The streaming pipeline is the code path that carries cleaned events from source files into every storage backend. It was already described at a structural level in Chapter 3 (section 3.4); this section covers the operational details.

The producer is a standalone Python process rather than a service with its own API. It reads the five CICIDS CSV files in sequence, pulls rows through `pd.read_csv` in chunks of 50,000 at a time, runs each chunk through the cleaning pipeline, and sends each cleaned row as a JSON message to the Kafka topic `network-events`. When all five files are exhausted, it publishes a sentinel message (`{"_sentinel": "END_OF_STREAM"}`) and exits. The sentinel is the signal the consumer uses to trigger final aggregations, so it has to be the last thing on the topic; the producer `flush`es the Kafka producer handle before sending it to guarantee ordering.

The consumer polls Kafka with a five-second timeout and accumulates messages into a 1,000-event batch. As described in section 3.4, every batch writes to all three backends in one iteration, with Neo4j handled on a background thread through a bounded queue (`queue.Queue(maxsize=50)`). The asymmetry between synchronous PostgreSQL and MongoDB writes and the background-thread Neo4j write reflects their relative throughput: Neo4j's `MERGE` with constraint checks writes at roughly 500 records per second, while PostgreSQL and MongoDB each handle around 2,000. Running Neo4j on the main loop would halve overall throughput. The bounded queue provides backpressure; if Neo4j falls more than 50 batches behind, the `enqueue` call blocks until the background thread catches up. Under sustained MAX-speed streaming the queue can drop batches rather than block indefinitely, which is the one place where the pipeline accepts lossy behaviour on the basis that

PostgreSQL remains the canonical store.

Every 10,000 events, the consumer runs the six batch aggregation scripts against PostgreSQL to refresh the summary tables during streaming. At end-of-stream (when the sentinel arrives), the consumer flushes the last partial batch, runs the summary scripts one final time, drains the Neo4j background queue, and calls `compute_co_attackers()` to derive the `CONNECTED_TO` relationships described in Chapter 5.

Running alongside the main pipeline, ksqlDB provides a streaming-SQL view over the same Kafka topic. Three tumbling-window tables are defined and materialised automatically at startup by the `ksqldb-init` service. The tables are `attack_rate_1min` (events per category in 1-minute windows, filtered to attacks), `protocol_rate_1min` (events per protocol in 1-minute windows), and `high_volume_sources` (source IPs generating more than 100 attack events in a 5-minute window). Listing 6.1 shows the first of these as a representative example. The dashboard reads these tables directly through ksqlDB's pull-query REST endpoint, which gives the Overview page live panels that refresh on their own schedule without touching PostgreSQL or MongoDB.

```sql
CREATE TABLE attack_rate_1min AS
SELECT
    attack_category,
    COUNT(*)      AS event_count,
    WINDOWSTART   AS window_start,
    WINDOWEND     AS window_end
FROM network_events_stream
WINDOW TUMBLING (SIZE 1 MINUTE)
WHERE attack_category <> 'Normal'
GROUP BY attack_category
EMIT CHANGES;
```

Listing 6.1: ksqlDB tumbling-window table that maintains a rolling one-minute attack rate per category.

As noted in Chapter 2, ksqlDB and the PostgreSQL summary tables answer different kinds of questions: ksqlDB tables refresh continuously as events arrive and cover the most recent windows, while summary tables cover the full history on a batch cadence.

6.3 Delivery Guarantees and Failure Handling

The pipeline has a handful of places where something can go wrong, and each one has a defined response. On the producer side, an unreadable row in a CSV file is skipped via pandas' `on_bad_lines='skip'` and logged, which lets a single bad line fail without aborting an entire file. On the consumer side, three failure classes are handled explicitly: PostgreSQL write failures, MongoDB write failures, and Neo4j write failures. They behave differently because their position in the pipeline differs.

PostgreSQL is written first in each batch, and it is the only backend whose write failure counts as a batch failure. If the PostgreSQL insert or dimension upsert throws an exception, the transaction rolls back and each event in the batch is sent to a dead-letter Kafka topic named `network-events-dlq`. The DLQ preserves the offending events for offline inspection rather than dropping them silently, and the consumer then moves on to the next batch. An empty DLQ after a full run is one of the clearest signals that the cleaning pipeline and the schema are in agreement with the input data; in the latest end-to-end run, the DLQ contained zero messages.

MongoDB write failures are treated as best-effort. The main `insert_many` uses `ordered=False`, so per-document errors (duplicate keys, for example) do not abort the batch. Broader failures (connection loss, auth failure) are logged but do not stop the consumer. This is defensible

because PostgreSQL is the canonical store, and MongoDB is an analytical mirror; the MongoDB content can be rebuilt from PostgreSQL after the fact if needed.

Neo4j failures are the most forgiving. Because the Neo4j write happens on a background thread with a bounded queue, failures on that thread are contained there and do not affect the main pipeline. If the queue fills up under sustained MAX-speed streaming, incoming batches are dropped with a warning rather than blocking the consumer indefinitely. This is the single explicit trade in the pipeline: Neo4j can lag behind or miss batches, and the pipeline accepts this rather than letting graph write speed gate relational and document throughput.

Restart behaviour follows from at-least-once semantics. On consumer crash, the Kafka consumer group retains the last committed offset, so the consumer resumes from the first batch that was not fully committed. Dimension upserts absorb the replay cleanly; fact rows and MongoDB documents may be duplicated for the re-processed batch; Neo4j relationship counts will double-count events in the re-processed batch. None of these is catastrophic at the dashboard level because all dashboard queries aggregate rather than count exact rows. In practice the consumer is wrapped in a retry loop that restarts it up to five times with a ten-second delay before giving up, which handles transient failures without operator intervention.

Chapter 7

Queries, Frontend, and User Scenarios

7.1 The Twelve Decision-Support Queries

The dashboard is backed by a catalogue of twelve decision-support queries, Q1 through Q12, each of which answers a recurring analytical question about the network intrusion data. Table 7.1 lists them.

Table 7.1: Decision-support query catalogue. Each query is implemented across the backends that can answer it naturally.

ID	Question	Primary summary	Backends
Q1	Attack count by category	`summary_attack_hourly`	All three
Q2	Hourly attack volume for a given day	`summary_attack_hourly`	All three
Q3	Top 10 source IPs by attack activity	`summary_top_sources`	All three
Q4	Attack-type distribution (share of traffic)	`summary_attack_hourly`	All three
Q5	Protocol usage breakdown for attack traffic	`summary_protocol_daily`	All three
Q6	Most targeted destination ports	`summary_dest_port_daily`	All three
Q7	Average flow duration by attack type	`summary_attack_hourly`	PG, Mongo
Q8	Co-attacking IP pairs (shared attack types)	`CONNECTED_TO` edges	Neo4j
Q9	Country-level threat summary	`summary_top_sources`	All three
Q10	Attack severity distribution over time	`summary_severity_daily`	All three
Q11	Weekend vs weekday attack comparison	`summary_weekend_weekday`	All three
Q12	Botnet activity timeline	`summary_attack_hourly`	All three

Each backend implements each query through the idiom native to it. PostgreSQL reads from the summary tables for most dashboard pages, falling back to fact-table JOINs only when the Backend Comparison page runs its fair-comparison variants. MongoDB reads from `attack_hourly_summary` for the queries that have a pre-aggregated equivalent (Q1, Q2, Q4, Q12) and uses aggregation pipelines over `network_events` for the rest. Neo4j reads counts directly off relationship properties, since the graph's edges already carry the aggregated values.

Two queries deserve individual mention because they sit outside the general pattern. Q7 (average flow duration) is answered only on PostgreSQL and MongoDB because the Neo4j graph does not store per-event duration; an answer on Neo4j would require storing every flow as a relationship property, which would undo the graph's aggregation benefit. Q8 (co-

attacking IP pairs) is answered only on Neo4j, where the `CONNECTED_TO` edges derived at end-of-stream make the answer a single two-hop match. The same question in PostgreSQL would need a self-join of the fact table on `attack_key`, and in MongoDB a `$lookup` against the same collection, both of which are expensive enough that the dashboard disables them rather than running them.

A separate set of fact-table variants exists on the PostgreSQL connector (`q1_attack_counts_fact` and friends) and is used only by the Backend Comparison page. These variants bypass the summary tables to scan `fact_network_event` with dimension JOINs, producing an engine-to-engine comparison between PostgreSQL and MongoDB at equal computational footing. Neo4j is not equalised the same way; its speed reflects its pre-aggregated model, and making it slower to match would falsify the design rather than reveal it. The comparison page therefore shows three points on a pre-computation spectrum, not three attempts at the same benchmark.

7.2 The Streamlit Dashboard

The dashboard uses Streamlit because it was the fastest way to build a Python-native interface that could talk to all three database drivers and ksqlDB's REST endpoint from the same process. The layout is sidebar-driven: eight pages stacked vertically, navigated through a button selector. Six of the pages are analytical surfaces over the warehouse data, the remaining two are operator-facing diagnostic views that surface the pipeline's runtime state.

The **Overview** page is the landing page. It shows four KPI cards (total events, attack events, unique attacking sources, and distinct attack categories), a bar chart of attack categories, an hourly-timeline line chart, and a pie chart of category share. It reads exclusively from PostgreSQL's `summary_attack_hourly`, and the Overview's live panels additionally pull from ksqlDB for the most recent tumbling-window values. The **Threat Profiling** page focuses on attackers: a ranked table of top source IPs by threat score, a country choropleth, a protocol-distribution chart, and a targeted-ports table. It reads mostly from `summary_top_sources` and `summary_dest_port_daily`.

The **Time Analysis** page answers the temporal questions: hourly and daily trend lines, severity distribution over time, a weekend-versus-weekday comparison, and a botnet-specific timeline. It uses `summary_attack_hourly`, `summary_severity_daily`, and `summary_weekend_weekday` The **Geo Analysis** page is a focused view on geography: a country-level threat summary, a map visualisation, and a table ranking countries by attack rate. The **Network Graph** page is the Neo4j page and the only one that reads from the graph backend: a co-attacker cluster visualisation, a top-attacked-destinations table, and a source-to-attack-type relationship view.

The **Backend Comparison** page is different from the other analytical pages. It runs the same query on PostgreSQL, MongoDB, and Neo4j in parallel and shows the results side by side along with a timing bar chart. A full-benchmark mode runs all twelve queries and produces a grouped timing chart and a summary table. This is the page where the three backends are actually compared in practice, and its timings form the basis of the quantitative observations in Chapter 8.

Two further pages provide operator visibility into the pipeline rather than analytical content. The **Logs** page tails the service log files that the `api` service aggregates, presenting a unified, filterable view across producer, consumer, and the databases, which is useful for diagnosing a stalled or misbehaving run without reaching into the Docker log streams. The **DLQ** page reads messages from the `network-events-dlq` Kafka topic through the same `api` service,

letting an operator inspect any events that the consumer rejected; during normal operation this page is empty, and the main use is after a schema or cleaning-rule change when something unexpected shows up.

7.3 User Scenarios

Two scenarios show how the dashboard is used in practice, one for each of the primary non-technical personas from Chapter 1.

Scenario A: SOC analyst morning review. An analyst opens the dashboard at the start of a shift and lands on the Overview page. The KPI cards and the hourly timeline give the current attack-volume shape at a glance, and any deviation from the usual daytime pattern (a spike in an off-hours bucket, for instance) is the first thing the analyst looks for. When something stands out, the next step is Threat Profiling: the top-sources table, sorted by threat score, surfaces the most active attackers for the period, and the destination-service breakdown indicates whether the traffic is concentrated on a single service or spread across many. If a single source is scanning many distinct destination ports, the primary-attack label on its threat-score row typically reads `PortScan`, which is the pattern the scenario is meant to illustrate. The analyst exports the relevant rows for the incident log and, if curious, opens Backend Comparison to run Q3 across the three backends, confirming that the ranking is consistent regardless of where the query is answered.

Scenario B: threat intelligence researcher studying a pattern. A researcher is looking at the weekly distribution of DoS attacks. From Time Analysis, the day-of-week chart shows DoS volume concentrated on Wednesday, which is the day the original CICIDS 2017 capture scheduled DoS attacks. The researcher switches to Threat Profiling, filters by category `DoS`, and examines the destination-service breakdown, where HTTP (port 80) dominates. The researcher then opens Backend Comparison and runs Q7 (average flow duration by attack type), which is available on PostgreSQL and MongoDB only. The result confirms a long-known DoS distinction: `DoS Hulk` produces very short flows on the order of tens of milliseconds, while `DoS Slowloris` produces long-lived flows on the order of several minutes per flow, and the two attack mechanics become visible from the duration distribution alone. The researcher exports the summary as a CSV for a longer report.

Chapter 8

Results and Verification

8.1 End-to-End Run

A full run means loading all five daily CSVs through the producer, letting the consumer drain Kafka and write to every backend, running the PostgreSQL batch jobs to refresh the six summary tables, waiting for the Neo4j background queue to drain, and finally executing `compute_co_attackers` to derive the `CONNECTED_TO` relationships. The pipeline is considered complete once all three backends reflect the end-of-stream state and no further writes are pending.

8.2 Data Volume and Integrity

All 2,099,971 rows from the five source CSVs survived the cleaning pipeline. The `vishwa132` variant had already been pre-filtered by its publisher to remove rows with missing critical fields and infinity-valued rate columns, so step 4 of the cleaning pipeline found nothing to drop on this input. Table 8.1 summarises the integrity checks performed against the loaded data.

Table 8.1: Data quality metrics after a full end-to-end run.

Metric	**Value**
Records loaded from source	2,099,971
Records dropped during cleaning	0
"Unknown" attack categories after mapping	0
Messages sent to the dead-letter queue	0
NULL timestamps after cleaning	0
NULL IPs after cleaning	0
Infinity values in numeric columns	0
Orphan fact rows (broken dimension foreign key)	0
PostgreSQL fact count = MongoDB event count	Yes (2,099,971 = 2,099,971)

Two of these results are worth calling out. The zero `Unknown` attack categories confirm that the multi-stage label normalisation in step 8 of the cleaning pipeline succeeded in mapping all twenty-seven label variants in the source data onto the nine curated categories. The exact equality between the PostgreSQL fact count and the MongoDB document count, after accounting for the background write path, confirms that both synchronous backends received every event the consumer processed.

8.3 Warehouse Statistics

The three backends populate to the row counts shown in Tables 8.2, 8.3, and 8.4.

Table 8.2: PostgreSQL star schema and summary tables.

Table	Rows	Grain
`fact_network_event`	2,099,971	One per flow event
`dim_time`	2,705	One per distinct timestamp
`dim_source`	411,658	One per (source_ip, port)
`dim_destination`	39,928	One per (dest_ip, port)
`dim_attack`	31	27 labels + 4 seed entries
`dim_protocol`	4	TCP, UDP, ICMP, OTHER
`summary_attack_hourly`	250	Hour × attack label
`summary_top_sources`	290	Day × source IP

Table 8.3: MongoDB collections.

Collection	Documents	Notes
`network_events`	2,099,971	Fully embedded documents
`attack_hourly_summary`	250	Mirror of PG hourly summary

Table 8.4: Neo4j graph: nodes and relationships after `compute_co_attackers`.

Node type	Count	Relationship	Count
SourceIP	174	ATTACKED	62,107
DestinationIP	19,044	TARGETED_BY	19,079
AttackType	31	CONNECTED_TO	15,051
TimeWindow	50	ACTIVE_IN	250
Protocol	4	USED_ATTACK	212
		USED_PROTOCOL	202
Total nodes	19,303	Total rel.	96,901

The Neo4j graph is much smaller than the PostgreSQL fact table, even though it represents the same underlying events. The 517,000 attack events compress into 62,107 `ATTACKED` edges, a reduction factor of roughly eight, which is the measurable consequence of shifting aggregation from query time to write time. Only 174 distinct source IPs actually launched attacks, a tiny fraction of the 411,658 source sockets in the PostgreSQL `dim_source` dimension; most source sockets generated benign traffic only, and the graph omits them by design.

8.4 Attack and Geographic Distribution

Table 8.5 reports the distribution of events across the nine categories the cleaning pipeline produces: one benign category (Normal) and eight attack categories. Just under a quarter of all traffic (24.6%) is malicious, with Reconnaissance (PortScan) and DoS making up the bulk of it.

Table 8.5: Event distribution by attack category. Severity is the assigned rating from the cleaning pipeline's mapping module.

Category	Events	Share	Severity
Normal (BENIGN)	1,582,561	75.4%	LOW
DoS	175,083	8.3%	HIGH
Reconnaissance (PortScan)	159,066	7.6%	LOW
DDoS	95,144	4.5%	CRITICAL
Infiltration	71,848	3.4%	CRITICAL
Brute Force	6,972	0.3%	MEDIUM
Botnet	4,803	0.2%	HIGH
Web Attack	2,056	0.1%	MEDIUM
Exploitation (Heartbleed)	11	<0.01%	CRITICAL

Attack intensity escalates through the week. Monday holds only benign traffic, establishing a baseline. Tuesday introduces brute-force attempts (6,972 events). Wednesday sees a significant escalation as DoS, web attacks, and infiltration activity begin (177,521 attack events). Thursday's infiltration campaign peaks at 71,767 events. Friday carries the largest attack volume of the week at 259,013 events, dominated by DDoS, PortScan, and Botnet activity. This shape matches the CICIDS 2017 capture schedule, which staged specific attack types on specific days rather than interleaving them.

Geographic results reflect the simulated geolocation (assumption A2). Vietnam dominates the top of the attack-by-country list with 441,714 attack events, followed by Russia at 72,829. The skew is a consequence of the deterministic IP-to-country hash mapping rather than real-world attacker distribution, and the simulation therefore serves as a sanity check for the pipeline rather than as a threat-intelligence finding in its own right.

8.5 Observed Query and Write Performance

Table 8.6 reports the write throughput observed during streaming on a standard laptop. The numbers are approximate because the real rate depends on background load, but the relative ordering is stable across runs.

Table 8.6: Observed write throughput per backend during streaming.

Backend	Write method	Throughput
PostgreSQL	Synchronous batch insert (1,000 rows)	∼2,000 records/s
MongoDB	Synchronous `insert_many` (1,000 docs)	∼2,000 records/s
Neo4j	Background thread, `MERGE` + `UNWIND` (500 rows)	∼500 records/s

The Neo4j bottleneck comes from `MERGE` constraint checks, which are unavoidable given the uniqueness constraints that keep the graph's node identity correct. The background thread prevents the slower write from pulling the main consumer loop down, and the bounded queue provides backpressure as described in Chapter 6.

Query-response times observed through the Backend Comparison page place the three backends on the pre-computation spectrum introduced in Chapter 5. PostgreSQL summary-table queries complete in the sub-second range because they read small pre-aggregated tables, while the fact-table variants used by the Backend Comparison page for fair benchmarking take noticeably longer because they scan the full fact table with dimension JOINs. MongoDB, when running aggregations over `network_events`, sits in a similar range to PostgreSQL's fact-table path, with its larger document size making full-collection scans marginally heavier. Neo4j reads counts directly from relationship properties and therefore responds fastest for its supported queries, at the cost of not being able to answer per-event questions such as Q7 at all. Exact measurements are available through the Backend Comparison page's full-benchmark mode, and the qualitative picture reported here is consistent across runs.

Chapter 9

Relational vs NoSQL Comparison

9.1 Framing the Comparison

A direct three-way benchmark would flatten the most interesting part of this project. PostgreSQL and MongoDB are comparable engine-to-engine: both store every raw event and aggregate at query time, so differences between them reflect normalisation versus embedding, indexing strategy, and the physical cost of scanning compact rows versus fat documents. Neo4j is on a different axis altogether, because it shifts aggregation from query time to write time (assumption A11 in Chapter 2). Its read speed therefore reflects a pre-aggregation strategy rather than a faster engine, and its missing columns on certain queries reflect what the graph no longer stores. The comparison in this chapter reads each backend against the analytical questions it is built to answer well, rather than against a uniform benchmark that would hide those distinctions.

9.2 Dimension-by-Dimension Comparison

Table 9.1 compares the three backends across the dimensions that matter for this workload.

Table 9.1: Backend comparison matrix. Qualitative labels reflect relative behaviour on this dataset and hardware profile.

Dimension	**PostgreSQL**	**MongoDB**	**Neo4j**
Schema	Rigid star schema	Flexible embedded documents	Property graph (nodes + edges)
Query language	SQL	Aggregation pipeline	Cypher
JOIN cost	Index-assisted, low	Not applicable (embedded)	Traversal, natural
Aggregation	Strong (`GROUP BY`, window)	Strong (`$group` pipeline)	Limited for full-scan aggregates
Relationship queries	Self-joins, expensive	`$lookup`, expensive	Pattern match, cheap
Storage shape	Normalised, compact	Denormalised, largest (∼3× PG)	Aggregated subgraph
Write speed (streaming)	Fast	Fast	Slow (`MERGE` constraint checks)
Read speed (summary-backed)	Sub-second	Sub-second (summary collection)	Fastest (pre-aggregated edges)
Read speed (full scan)	Multi-second (fact-table JOINs)	Multi-second (∼3× rows scanned)	Not applicable
Per-event detail	Preserved	Preserved	Not preserved
ACID	Yes	Tunable	Yes
Unique strength	Multi-dimensional aggregation	Schemaless flexibility, single-doc reads	Relationship and pattern queries
Unique weakness	Verbose multi-table JOINs	No cross-collection JOIN (only `$lookup`)	Cannot answer per-event questions

Two rows in the table drive most of the practical difference. **Storage shape** and **per-event detail** together determine which questions each backend can answer and at what cost: PostgreSQL pays in JOIN verbosity to get compact storage, MongoDB pays in storage size to get join-free event reads, and Neo4j pays in the loss of per-event detail to get pre-computed relationship counts. **Read speed (summary-backed)** versus **read speed (full scan)** shows that the relational warehouse has two operating modes: a fast mode when its summary tables cover the question, and a slower full-scan mode when they do not. The Backend Comparison page deliberately uses the slower mode to benchmark PostgreSQL against MongoDB fairly; in production dashboard use, the summary mode is the one that matters.

9.3 Query Suitability

Not every query is answerable on every backend. Table 9.2 records which backend is native, which can answer the question with some compromise, and which is excluded.

Table 9.2: Query suitability by backend. "Native" means the backend answers the question through its natural idiom; "Partial" means it answers but with fewer columns or an approximation; "–" means the query is not implemented for that backend.

ID	Query	PG	Mongo	Neo4j
Q1	Attack count by category	Native	Native	Native
Q2	Hourly attack volume	Native	Native	Native
Q3	Top source IPs	Native	Native	Partial
Q4	Attack-type distribution	Native	Native	Native
Q5	Protocol usage breakdown	Native	Native	Partial
Q6	Most targeted destination ports	Native	Native	Partial
Q7	Average flow duration by attack type	Native	Native	–
Q8	Co-attacking IP pairs	–	–	Native
Q9	Country-level threat summary	Native	Native	Native
Q10	Severity distribution over time	Native	Native	Native
Q11	Weekend vs weekday comparison	Native	Native	Native
Q12	Botnet activity timeline	Native	Native	Native

The table captures the asymmetry of the three-backend design. Q7 is the clearest example of what the graph gives up: average flow duration depends on per-event durations that Neo4j does not store, so the backend has no way to answer. Q8 is the clearest example of what the graph gives back: two source IPs that share attack types are found by a two-hop pattern match in Cypher, while the same question in PostgreSQL needs an expensive self-join of the fact table on attack key and in MongoDB needs a cross-collection `$lookup`, both of which are expensive enough that the dashboard disables them rather than running them. The "Partial" entries in Neo4j's column mark queries where the graph answers but with fewer columns: Q3 returns source ranking without the `threat_score` or distinct-target count that PostgreSQL's summary carries, Q5 returns protocol totals without the attack/normal split, and Q6 returns top attacked destinations rather than top attacked ports, because the graph models `DestinationIP` rather than `(dest_ip, dest_port)` sockets.

9.4 Advantages and Disadvantages

PostgreSQL is the canonical store for this project. It combines ACID guarantees, SQL's expressive power for multi-dimensional aggregation, and pre-aggregated summary tables that make dashboard queries sub-second even on a laptop. Its weakness is the verbosity of the relational model: answering a question about a single event requires joining five dimension tables, and relationship-oriented questions (the shape of Q8) are awkward rather than natural. At this project's scale the trade is clearly worth it, and PostgreSQL ends up being the one backend that can answer every query that is not graph-native.

MongoDB excels at questions that centre on an event and its context, because the embedded document model returns everything the dashboard needs for a single-event view in one read. Its aggregation pipeline handles most analytical queries expressively, and the schemaless model makes it the most forgiving of the three to future changes in the log schema. Its weakness is full-collection aggregation: the documents are roughly three times larger than the corresponding PostgreSQL fact rows, so the same `$group` scan moves more data through memory than the relational path. It is also the backend that depends most heavily on index planning; without the right compound index, a full-collection scan is the slowest path through the warehouse.

Neo4j is uniquely capable of questions that are shaped as relationships. Q8 is the headline

example, but the same property extends to anything asking how entities connect: which source IPs target the same destinations, which attack types co-occur on the same source, which protocols correlate with which categories. Its weakness is the mirror of this strength: questions that need per-event detail (Q7) or full-scan aggregates across every event cannot be answered on the graph at all, because the events themselves are not stored. The graph's write speed is also the pipeline's slowest path, because `MERGE` with uniqueness constraints is expensive relative to PostgreSQL's append-only inserts or MongoDB's ordered-false bulk writes.

9.5 Synthesis

No single backend wins outright. PostgreSQL is the strongest general-purpose store, MongoDB is the most flexible shape for event-centric reads, and Neo4j is the only one that makes relationship queries cheap. The three together cover a wider analytical surface than any one of them alone, which is the point of building all three on top of the same cleaned event stream rather than choosing among them. A production system would usually pick one, and the right choice would depend on whether the dominant question is aggregate ("how much of what kind"), event-centric ("what happened in this flow"), or relational ("which things are connected to which"). For this project, the value sits in seeing how the same data reshapes itself when asked the same questions through three different models.

Chapter 10

Conclusion

10.1 Summary of Findings

The project built a three-backend data warehouse over the CICIDS 2017 intrusion dataset, fed by a single cleaning pipeline and a Kafka-based streaming layer that wrote to all three stores simultaneously. PostgreSQL carried the canonical star schema with pre-aggregated summary tables; MongoDB carried the same events as embedded documents with an incrementally-maintained summary collection; Neo4j carried an aggregated property graph in which the relationship properties held pre-computed counts. The twelve decision-support queries ran across all three where feasible, and the backend comparison surfaced the genuine architectural trade-offs rather than a uniform benchmark. The clearest finding is that the three models are complementary rather than competing: PostgreSQL is the canonical aggregator, MongoDB is the idiomatic store for event-centric reads, and Neo4j is the one that makes relationship queries cheap enough to be routine.

10.2 Limitations

Several simplifications frame how the results should be read. The geolocation attached to every source IP is simulated from a deterministic hash, so country-level findings are a sanity check for the pipeline rather than threat intelligence. Timestamps were reconstructed from the filename and a uniform distribution across a working window because the `vishwa132` variant truncates them to minute-second precision. The deployment is single-node, so none of the observed throughput figures generalise to a clustered Kafka or a sharded PostgreSQL, the same is true of the Neo4j write speed, which is dominated by constraint checks that a cluster would distribute. The dataset's attack labels are trusted as ground truth, even though in a real system those labels would themselves come from a classifier and carry their own error. Finally, there is no authentication on any service, so the system would need more work before it could face a real network.

10.3 Closing

The data-warehouse work that typically stops at a single relational schema becomes more interesting when two alternative implementations sit alongside it, because the contrast makes each model's assumptions visible. CyberSight DW held that contrast together in one running system, and the twelve queries serve as the instrument that reads each backend on its own terms.

Bibliography

1. Hessnawi, Yasin. *CyberSight DW: Source Code Repository*. GitHub. https://github.com/yasinhessnawi1/CyberSight_DW. 2026.

2. Sharafaldin, Iman, Arash Habibi Lashkari, and Ali A. Ghorbani. "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization." In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. Funchal, Portugal: SCITEPRESS, Jan. 2018, pp. 108–116.

3. vishwa132. *CICIDS-2017 Dataset*. Hugging Face Datasets. https://huggingface.co/datasets/vishwa132/CICIDS-2017. 2024.

Appendix A

Full SQL Listings

This appendix reproduces the SQL artefacts referenced throughout the report. Section A.1 lists the four summary-table DDLs not shown inline in Chapter 4. Section A.2 lists the four batch scripts not shown inline in Chapter 4 section 4.6. Section A.3 contains the dimension seed script. Section A.4 contains the full ksqlDB initialisation script. Section A.5 contains the twelve reference SQL files for the decision-support queries Q1 through Q12.

A.1 Additional Summary Table DDL

```sql
CREATE TABLE summary_protocol_daily (
    summary_id      SERIAL PRIMARY KEY,
    day_bucket      DATE NOT NULL,
    protocol_name   VARCHAR(20) NOT NULL,
    total_events    BIGINT NOT NULL DEFAULT 0,
    attack_events   BIGINT NOT NULL DEFAULT 0,
    attack_ratio    NUMERIC(5,4) DEFAULT 0,
    UNIQUE (day_bucket, protocol_name)
);

CREATE INDEX idx_summary_protocol_day ON summary_protocol_daily(day_bucket);
```

Listing A.1: DDL for `summary_protocol_daily`.

```sql
CREATE TABLE summary_severity_daily (
    summary_id      SERIAL PRIMARY KEY,
    day_bucket      DATE NOT NULL,
    severity        VARCHAR(10) NOT NULL,
    event_count     BIGINT NOT NULL DEFAULT 0,
    unique_sources  INTEGER DEFAULT 0,
    created_at      TIMESTAMP DEFAULT NOW(),
    UNIQUE (day_bucket, severity)
);

CREATE INDEX idx_summary_severity_day ON summary_severity_daily(day_bucket);
```

Listing A.2: DDL for `summary_severity_daily`.

```sql
CREATE TABLE summary_dest_port_daily (
    summary_id      SERIAL PRIMARY KEY,
    day_bucket      DATE NOT NULL,
    dest_port       INTEGER NOT NULL,
    service_name    VARCHAR(50) DEFAULT 'Unknown',
    attack_category VARCHAR(50) NOT NULL,
    event_count     BIGINT NOT NULL DEFAULT 0,
    created_at      TIMESTAMP DEFAULT NOW(),
    UNIQUE (day_bucket, dest_port, attack_category)
```

```
);

CREATE INDEX idx_summary_dest_port_day ON summary_dest_port_daily(day_bucket);
```

Listing A.3: DDL for summary_dest_port_daily.

```
CREATE TABLE summary_weekend_weekday (
    summary_id      SERIAL PRIMARY KEY,
    is_weekend      BOOLEAN NOT NULL,
    attack_category VARCHAR(50) NOT NULL,
    total_events    BIGINT NOT NULL DEFAULT 0,
    attack_events   BIGINT NOT NULL DEFAULT 0,
    attack_rate_pct NUMERIC(5,2) DEFAULT 0,
    created_at      TIMESTAMP DEFAULT NOW(),
    UNIQUE (is_weekend, attack_category)
);

CREATE INDEX idx_summary_weekend ON summary_weekend_weekday(is_weekend);
```

Listing A.4: DDL for summary_weekend_weekday.

A.2 Additional Batch Scripts

```
-- Batch Job: Populate summary_protocol_daily
-- Run after each load of fact_network_event

INSERT INTO summary_protocol_daily
    (day_bucket, protocol_name, total_events, attack_events, attack_ratio)
SELECT
    date_trunc('day', dt.timestamp)::DATE  AS day_bucket,
    dp.protocol_name,
    COUNT(*)                               AS total_events,
    COUNT(*) FILTER (WHERE f.is_attack = TRUE) AS attack_events,
    ROUND(
        COUNT(*) FILTER (WHERE f.is_attack = TRUE)::NUMERIC
        / NULLIF(COUNT(*), 0), 4
    )                                      AS attack_ratio
FROM fact_network_event f
JOIN dim_time     dt ON f.time_key     = dt.time_key
JOIN dim_protocol dp ON f.protocol_key = dp.protocol_key
GROUP BY date_trunc('day', dt.timestamp)::DATE, dp.protocol_name
ON CONFLICT (day_bucket, protocol_name)
DO UPDATE SET
    total_events  = EXCLUDED.total_events,
    attack_events = EXCLUDED.attack_events,
    attack_ratio  = EXCLUDED.attack_ratio;
```

Listing A.5: batch_protocol_daily.sql: populates summary_protocol_daily.

```
-- Batch Job: Populate summary_severity_daily
-- Run after each load of fact_network_event

INSERT INTO summary_severity_daily
    (day_bucket, severity, event_count, unique_sources)
SELECT
    date_trunc('day', dt.timestamp)::DATE  AS day_bucket,
    da.severity,
    COUNT(*)                               AS event_count,
    COUNT(DISTINCT f.source_key)           AS unique_sources
FROM fact_network_event f
JOIN dim_time   dt ON f.time_key   = dt.time_key
JOIN dim_attack da ON f.attack_key = da.attack_key
WHERE f.is_attack = TRUE
GROUP BY date_trunc('day', dt.timestamp)::DATE, da.severity
```

```sql
ON CONFLICT (day_bucket, severity)
DO UPDATE SET
    event_count     = EXCLUDED.event_count,
    unique_sources  = EXCLUDED.unique_sources,
    created_at      = NOW();
```

Listing A.6: `batch_severity_daily.sql`: populates `summary_severity_daily`.

```sql
-- Batch Job: Populate summary_dest_port_daily
-- Run after each load of fact_network_event

INSERT INTO summary_dest_port_daily
    (day_bucket, dest_port, service_name, attack_category, event_count)
SELECT
    date_trunc('day', dt.timestamp)::DATE  AS day_bucket,
    dd.dest_port,
    dd.service_name,
    da.attack_category,
    COUNT(*)                               AS event_count
FROM fact_network_event f
JOIN dim_time        dt ON f.time_key    = dt.time_key
JOIN dim_destination dd ON f.dest_key    = dd.dest_key
JOIN dim_attack      da ON f.attack_key  = da.attack_key
WHERE f.is_attack = TRUE
GROUP BY date_trunc('day', dt.timestamp)::DATE, dd.dest_port, dd.service_name, da.
    attack_category
ON CONFLICT (day_bucket, dest_port, attack_category)
DO UPDATE SET
    service_name = EXCLUDED.service_name,
    event_count  = EXCLUDED.event_count,
    created_at   = NOW();
```

Listing A.7: `batch_dest_port.sql`: populates `summary_dest_port_daily`.

```sql
-- Batch Job: Populate summary_weekend_weekday
-- Run after each load of fact_network_event

INSERT INTO summary_weekend_weekday
    (is_weekend, attack_category, total_events, attack_events, attack_rate_pct)
SELECT
    dt.is_weekend,
    da.attack_category,
    COUNT(*)                                      AS total_events,
    COUNT(*) FILTER (WHERE f.is_attack = TRUE)    AS attack_events,
    ROUND(
        COUNT(*) FILTER (WHERE f.is_attack = TRUE) * 100.0 / NULLIF(COUNT(*), 0),
    2
    )                                             AS attack_rate_pct
FROM fact_network_event f
JOIN dim_time   dt ON f.time_key    = dt.time_key
JOIN dim_attack da ON f.attack_key  = da.attack_key
GROUP BY dt.is_weekend, da.attack_category
ON CONFLICT (is_weekend, attack_category)
DO UPDATE SET
    total_events    = EXCLUDED.total_events,
    attack_events   = EXCLUDED.attack_events,
    attack_rate_pct = EXCLUDED.attack_rate_pct,
    created_at      = NOW();
```

Listing A.8: `batch_weekend.sql`: populates `summary_weekend_weekday`.

A.3 Dimension Seed Data

```sql
-- Seed dim_protocol (covers all IANA numbers observed in CICIDS 2017)
INSERT INTO dim_protocol (protocol_name, transport_layer) VALUES
    ('TCP',        'Transport'),
    ('UDP',        'Transport'),
    ('SCTP',       'Transport'),
    ('ICMP',       'Network'),
    ('ICMPv6',     'Network'),
    ('IGMP',       'Network'),
    ('HOPOPT',     'Network'),
    ('IPv6',       'Network'),
    ('IPv6-Route', 'Network'),
    ('IPv6-Frag',  'Network'),
    ('IPv6-NoNxt', 'Network'),
    ('IPv6-Opts',  'Network'),
    ('GRE',        'Network'),
    ('ESP',        'Network'),
    ('AH',         'Network'),
    ('OSPF',       'Network'),
    ('PIM',        'Network'),
    ('VRRP',       'Network'),
    ('OTHER',      'Unknown');

-- Seed dim_attack with all known CICIDS 2017 labels
INSERT INTO dim_attack (attack_label, attack_category, severity, description) VALUES
    ('BENIGN',                        'Normal',         'LOW',      'Normal network traffic'),
    ('DoS Hulk',                      'DoS',            'HIGH',     'HTTP GET flood using Hulk tool'),
    ('DoS GoldenEye',                 'DoS',            'HIGH',     'HTTP DoS using GoldenEye tool'),
    ('DoS slowloris',                 'DoS',            'MEDIUM',   'Slow HTTP header attack'),
    ('DoS Slowhttptest',              'DoS',            'MEDIUM',   'Slow HTTP body attack'),
    ('DDoS',                          'DDoS',           'CRITICAL', 'Distributed Denial of Service'),
    ('DDoS LOIT',                     'DDoS',           'CRITICAL', 'DDoS Low Orbit Ion Cannon'),
    ('PortScan',                      'Reconnaissance', 'LOW',      'Port scanning activity'),
    ('FTP-Patator',                   'Brute Force',    'MEDIUM',   'FTP credential brute force'),
    ('SSH-Patator',                   'Brute Force',    'MEDIUM',   'SSH credential brute force'),
    ('Bot',                           'Botnet',         'HIGH',     'Botnet activity'),
    ('Web Attack -- Brute Force',     'Web Attack',     'MEDIUM',   'Web form brute force'),
    ('Web Attack -- XSS',             'Web Attack',     'MEDIUM',   'Cross-site scripting attack'),
    ('Web Attack -- Sql Injection',   'Web Attack',     'HIGH',     'SQL injection attempt'),
    ('Infiltration',                  'Infiltration',   'CRITICAL', 'Network infiltration attempt'),
    ('Infiltration - Portscan',       'Infiltration',   'CRITICAL', 'Infiltration via port scanning'),
    ('Infiltration - CoolDisc',       'Infiltration',   'CRITICAL', 'Infiltration via CoolDisc'),
    ('Infiltration - Dropbox',        'Infiltration',   'CRITICAL', 'Infiltration via Dropbox download'),
    ('Heartbleed',                    'Exploitation',   'CRITICAL', 'OpenSSL Heartbleed exploit'),
    ('DoS Slowloris',                 'DoS',            'MEDIUM',   'Slow HTTP header attack (case variant)'),
```

```
    ('DoS Hulk - Attempted',            'DoS',            'HIGH',     'Attempted DoS Hulk attack'),
    ('DoS GoldenEye - Attempted',       'DoS',            'HIGH',     'Attempted DoS GoldenEye attack'),
    ('DoS Slowhttptest - Attempted',    'DoS',            'MEDIUM',   'Attempted slow HTTP body attack'),
    ('DoS Slowloris - Attempted',       'DoS',            'MEDIUM',   'Attempted slow HTTP header attack'),
    ('FTP-Patator - Attempted',         'Brute Force',    'MEDIUM',   'Attempted FTP brute force'),
    ('SSH-Patator - Attempted',         'Brute Force',    'MEDIUM',   'Attempted SSH brute force');
```

Listing A.9: `seed_data.sql`: protocol and attack dimension seeds, auto-executed by the PostgreSQL container on first start.

A.4 ksqlDB Initialisation

```
-- Stream over the raw Kafka topic
CREATE STREAM IF NOT EXISTS network_events_stream (
    flow_id          VARCHAR,
    source_ip        VARCHAR,
    source_port      INTEGER,
    destination_ip   VARCHAR,
    destination_port INTEGER,
    protocol         VARCHAR,
    timestamp        VARCHAR,
    flow_duration    BIGINT,
    fwd_packets      INTEGER,
    bwd_packets      INTEGER,
    fwd_bytes        BIGINT,
    bwd_bytes        BIGINT,
    flow_bytes_per_sec  DOUBLE,
    flow_packets_per_sec DOUBLE,
    label            VARCHAR,
    attack_category  VARCHAR,
    severity         VARCHAR,
    source_country   VARCHAR,
    source_region    VARCHAR
) WITH (
    KAFKA_TOPIC  = 'network-events',
    VALUE_FORMAT = 'JSON'
);

-- 1-minute tumbling window: attack rate per category
CREATE TABLE IF NOT EXISTS attack_rate_1min AS
SELECT
    attack_category,
    COUNT(*)        AS event_count,
    WINDOWSTART     AS window_start,
    WINDOWEND       AS window_end
FROM network_events_stream
WINDOW TUMBLING (SIZE 1 MINUTE)
WHERE attack_category <> 'Normal'
GROUP BY attack_category
EMIT CHANGES;

-- 1-minute tumbling window: events per protocol
CREATE TABLE IF NOT EXISTS protocol_rate_1min AS
SELECT
    protocol,
    COUNT(*)        AS event_count,
    WINDOWSTART     AS window_start
FROM network_events_stream
```

```sql
WINDOW TUMBLING (SIZE 1 MINUTE)
GROUP BY protocol
EMIT CHANGES;

-- 5-minute tumbling window: high-volume attack sources (> 100 events)
CREATE TABLE IF NOT EXISTS high_volume_sources AS
SELECT
    source_ip,
    COUNT(*)        AS attack_count,
    WINDOWSTART     AS window_start
FROM network_events_stream
WINDOW TUMBLING (SIZE 5 MINUTES)
WHERE attack_category <> 'Normal'
GROUP BY source_ip
HAVING COUNT(*) > 100
EMIT CHANGES;
```

Listing A.10: `ksql_init.sql`: full ksqlDB stream and tumbling-window tables, applied automatically by the `ksqldb-init` service on startup.

A.5 Decision-Support Query Reference SQL

The twelve files below sit in `sql/queries/` in the repository. They document the canonical question each query answers, with the PostgreSQL implementation expressed against the summary tables where possible. The dashboard's MongoDB and Neo4j connectors carry equivalent implementations in their native idioms.

```sql
-- Q1: Total attack count by category
-- Backend: All | Uses Summary: Yes

SELECT
    attack_category,
    SUM(event_count) AS total_events
FROM summary_attack_hourly
WHERE attack_category <> 'Normal'
GROUP BY attack_category
ORDER BY total_events DESC;
```

Listing A.11: `Q1_attack_counts.sql`

```sql
-- Q2: Hourly attack volume trend for a given day
-- Backend: All | Uses Summary: Yes
-- Parameter: target_date (e.g. '2017-07-05')

SELECT
    hour_bucket,
    attack_category,
    SUM(event_count) AS event_count
FROM summary_attack_hourly
WHERE hour_bucket::date = :target_date
  AND attack_category <> 'Normal'
GROUP BY hour_bucket, attack_category
ORDER BY hour_bucket;
```

Listing A.12: `Q2_hourly_trend.sql`

```sql
-- Q3: Top 10 source IPs by attack count
-- Backend: All | Uses Summary: Yes

SELECT
    source_ip,
    country,
    SUM(attack_count)    AS total_attacks,
```

```sql
    SUM(distinct_targets) AS total_targets,
    MAX(threat_score)     AS max_threat_score
FROM summary_top_sources
WHERE attack_count > 0
GROUP BY source_ip, country
ORDER BY total_attacks DESC
LIMIT 10;
```

Listing A.13: Q3_top_sources.sql

```sql
-- Q4: Attack type distribution (BENIGN vs attacks)
-- Backend: All | Uses Summary: Yes (summary_attack_hourly)

SELECT
    attack_category,
    SUM(event_count) AS event_count,
    ROUND(SUM(event_count) * 100.0 / SUM(SUM(event_count)) OVER (), 2) AS pct
FROM summary_attack_hourly
GROUP BY attack_category
ORDER BY event_count DESC;
```

Listing A.14: Q4_attack_distribution.sql

```sql
-- Q5: Protocol usage breakdown
-- Backend: All | Uses Summary: Yes (summary_protocol_daily)

SELECT
    protocol_name,
    SUM(total_events)  AS total_events,
    SUM(attack_events) AS attack_events,
    ROUND(
        SUM(attack_events)::NUMERIC * 100.0 / NULLIF(SUM(total_events), 0), 2
    ) AS attack_pct
FROM summary_protocol_daily
GROUP BY protocol_name
ORDER BY total_events DESC;
```

Listing A.15: Q5_protocol_breakdown.sql

```sql
-- Q6: Most targeted destination ports
-- Backend: All | Uses Summary: Yes (summary_dest_port_daily)

SELECT
    dest_port,
    service_name,
    SUM(event_count) AS attack_count
FROM summary_dest_port_daily
GROUP BY dest_port, service_name
ORDER BY attack_count DESC
LIMIT 15;
```

Listing A.16: Q6_targeted_ports.sql

```sql
-- Q7: Average flow duration by attack type
-- Backend: All | Uses Summary: Yes (summary_attack_hourly)

SELECT
    attack_label,
    attack_category,
    SUM(event_count) AS event_count,
    ROUND(SUM(avg_duration_ms * event_count) / NULLIF(SUM(event_count), 0), 2) AS
    avg_duration_ms,
    ROUND(SUM(total_bytes)::NUMERIC / NULLIF(SUM(event_count), 0), 2) AS
    avg_total_bytes
FROM summary_attack_hourly
WHERE attack_category <> 'Normal'
```

```sql
GROUP BY attack_label, attack_category
ORDER BY avg_duration_ms DESC;
```

Listing A.17: Q7_avg_duration.sql

```sql
-- Q8: Co-attacking IPs (shared attack type)
-- Backend: Neo4j only | Uses Summary: No
-- This query is best expressed in Cypher (see Neo4j connector).
-- PostgreSQL equivalent below is for reference only.

SELECT
    ds1.source_ip AS ip1,
    ds2.source_ip AS ip2,
    da.attack_label AS shared_attack,
    COUNT(*) AS co_count
FROM fact_network_event f1
JOIN fact_network_event f2
    ON f1.attack_key = f2.attack_key
    AND f1.source_key < f2.source_key
JOIN dim_source ds1 ON f1.source_key = ds1.source_key
JOIN dim_source ds2 ON f2.source_key = ds2.source_key
JOIN dim_attack da  ON f1.attack_key = da.attack_key
WHERE f1.is_attack = TRUE
GROUP BY ds1.source_ip, ds2.source_ip, da.attack_label
ORDER BY co_count DESC
LIMIT 20;
```

Listing A.18: Q8_co_attackers.sql: the project's primary implementation is Cypher on Neo4j; the SQL version shown here is a reference self-join for comparison.

```sql
-- Q9: Country-level threat summary
-- Backend: All | Uses Summary: Yes (summary_top_sources)

SELECT
    country,
    SUM(attack_count) AS attack_events,
    COUNT(DISTINCT source_ip) AS unique_ips,
    ROUND(AVG(threat_score), 2) AS avg_threat_score
FROM summary_top_sources
GROUP BY country
ORDER BY attack_events DESC;
```

Listing A.19: Q9_country_summary.sql

```sql
-- Q10: Attack severity distribution over time
-- Backend: All | Uses Summary: Yes (summary_severity_daily)

SELECT
    day_bucket AS day,
    severity,
    event_count
FROM summary_severity_daily
ORDER BY day_bucket, severity;
```

Listing A.20: Q10_severity_over_time.sql

```sql
-- Q11: Weekend vs weekday attack comparison
-- Backend: All | Uses Summary: Yes (summary_weekend_weekday)

SELECT
    is_weekend,
    attack_category,
    total_events,
    attack_events,
    attack_rate_pct
FROM summary_weekend_weekday
```

```sql
ORDER BY is_weekend, attack_events DESC;
```

Listing A.21: Q11_weekend_weekday.sql

```sql
-- Q12: Botnet activity timeline
-- Backend: All | Uses Summary: Yes (summary_attack_hourly)

SELECT
    hour_bucket,
    SUM(event_count) AS bot_events,
    SUM(unique_sources) AS unique_bot_sources,
    SUM(total_bytes) AS total_bytes
FROM summary_attack_hourly
WHERE attack_category = 'Botnet'
GROUP BY hour_bucket
ORDER BY hour_bucket;
```

Listing A.22: Q12_botnet_timeline.sql

Appendix B

AI Declaration

Have AI-based tools been used in the preparation of this report/thesis?

☐ No ☑ Yes

If no, sign below. If yes: specify type of tool and area of use below.

Text

☑ **Spell checking.**
Have parts of the text been checked by: Grammarly, Ginger, Grammarbot, LanguageTool, ProWritingAid, Sapling, Trinka.ai or similar tools?

☐ **Text generation.**
Have parts of the text been generated by: ChatGPT, GrammarlyGO, Copy.AI, WordAi, WriteSonic, Jasper, Simplified, Rytr or similar tools?

☑ **Writing assistance.**
Have one or more of the ideas or approaches in the assignment been suggested by: ChatGPT, Google Bard, Bing chat, YouChat or similar tools?

If yes to use of a text tool - specify the use here:

ChatGPT provided by UiO (another university) was used for sentence reformulation, English word choice and formulation improvements. Writing assistance was minimal, used for difficult concepts and parts that were hard to phrase manually. Both UiO ChatGPT and Overleaf AI were used for this purpose.

Code and algorithms

☑ **Programming assistance.**
Have parts of the code/algorithms that i) appear directly in the report or ii) have been used for production of results such as figures, tables or numerical values been generated by: GitHub Copilot, CodeGPT, Google Codey/Studio Bot, Replit Ghostwriter, Amazon CodeWhisperer, GPT Engineer, ChatGPT, Google Bard or similar tools?

If yes to use of a programming tool - specify the use here:

GitHub Copilot in VS Code was used for code line completion and debugging. AI from the

tab chat was used for debugging code. Pseudo code and example code were provided by AI when required.

Images and figures

☐ **Image generation.**
Have one or more of the images/figures in the report been generated by: Midjourney, Jasper, WriteSonic, Stability AI, Dall-E or similar tools?

If yes to use of an image tool - specify the use here:

Not applicable. All figures in the report are hand-written TikZ diagrams compiled directly in LaTeX. No image-generation tools were used.

Other AI tools. Have other types of tools been used? If yes, specify the use here:

Not applicable.

☑ I am familiar with UiA's regulations for the use of artificial intelligence. I have accounted for all use of artificial intelligence either i) directly in the report or ii) in this form.

Grimstad 2026
Signature/Date/Place